DI31013 – Engineering Design Project

Design and Implementation of a Multifunctional Intelligent Logistics Transport Vehicle Based on mbed LPC1768 Microcontroller

Group 2

Yuang Zhou	Junyuan Qin	Shucen Guo	Jin Li	Yuqiu Zhang
2617376	2617399	2617512	2617448	2617481
Microcontroller Implementation Software Development and Function Realization		Mechanical Design Mechanical Assembly and Operation Analyse Results and Discussion Future Work		Electrical Circuit and PCB Design
Abstract Introduction Conclusion				

Content

Abstract

In response to the growing demand for automation in short-range logistics, this project designs and implements a multifunctional intelligent transport vehicle based on an mbed LPC1768 microcontroller. The vehicle integrates omnidirectional Mecanum wheels and multiple functions including Bluetooth wireless remote control, ultrasonic obstacle detection with automated avoidance and parking, smooth stopping, autonomous following, path recording with forward and reverse replay, as well as an electric tipping-bucket unloading mechanism. A modular hardware architecture is adopted, and the embedded software is structured as a Finite-State Machine (FSM) with parallel multithreaded tasks to coordinate sensing and control. The design process involved mechanical modeling with finite-element analysis for structural integrity, PCB and circuit design for motor power and IIC communication, and firmware development for motor and sensor control. Comprehensive simulation and physical trials testing, verified that all intended functions meet performance requirements: unit-level tests confirmed stable operation of each module, and system-level integration tests demonstrated precise path replay, reliable obstacle avoidance with smooth deceleration under inertia, and millisecond-level Bluetooth control response without lag. The completed prototype is in compliance with the initial design aims, which proves the efficiency of the built-in solution as a low-cost automated short-distance logistics transport vehicle.

1 Introduction

With the in-depth integration of Industry 4.0 and smart manufacturing and with it, the sustained progress of intelligent logistics as the central node that links the decision-making between production and warehousing is rapidly moving toward automation, flexibility, and integration [1]. Statistics show that the market scale of domestic intelligent logistics equipment has reached more than 21 billion dollar in 2025, with short-haul material transport equipment scenarios, covering workshops and warehouses, taking more than 35 percent in the market. Nonetheless, the classical tools of transport are usually associated with such drawbacks as they are manual-oriented, fail to detect the environmental perception, and possess functionality at a single time [2]. To illustrate, manual tasks are fraught with poor efficiency and intense labor, whereas simple electronic trolleys can hardly fulfill the functions of the automatic obstacle avoidance and self-planning of the paths, not being capable to provide the efficient cooperation in the modern production [3].

Embedded technology and multi-sensor fusion development offers a great solution to solve the mentioned problems. The high level of integration, highability of the peripheral expansion of mbed LPC1768 microcontroller and the superbness of the real-time execution has made it the core control unit of smart mobile equipment [4]. In combination with other technologies like ultrasonic ranging, Bluetooth wireless communications, and the H-bridge motor drive, it can be used to achieve environmental perception, wireless control, and autonomous decision-making of logistics automation vehicles, The solution is lightweight to be applied to short-range automation of logistics [5].

Based on this, this project designs and implements a multifunctional intelligent logistics transport vehicle, with the mbed LPC1768 as the core controller. It integrates Mecanum wheel omnidirectional movement, Bluetooth remote control, ultrasonic automatic obstacle avoidance, smooth stop, auto-follow, path memory and reverse playback, and automatic tipping bucket unloading. The vehicle adopts a modular hardware architecture and Finite State Machine (FSM) software design, balancing structural compactness and functional integration, and can be adapted to multi-scenario applications such as workshop material transfer and warehouse goods distribution. The project aims to improve the automation level and operational convenience of short-distance logistics transportation through the optimization of hardware integration and innovation of software algorithms, providing technical reference for the research and development of low-cost intelligent logistics equipment.

2 Mechanical Design

2.1 Mechanical Design Description

The intelligent logistics transport vehicle of the project is a differential-driven platform that uses the mbed LPC1768 as its primary controller to perform (but is not limited to) the following core functions: electric tipping bucket unloading, Bluetooth remote control, ultrasonic obstacle detection and parking, movement along a predetermined path and auto-following. Stable structure, ease of electronics and sensor integration, and ease of production and maintenance are the goals of the mechanical design. The following Table 1 lists the primary design specifications.

Table 1. Main design specifications of the vehicle.

Vehicle body (Length × Width × Height)	339.0 mm×240.8 mm× 193.9 mm
Wheelbase and Track width	221.1 mm, 210.3mm
Preset No-load weight and Rated load	~2.0 kg, ~2.0 kg
Operating voltage	11.1 Volts, 5.0 Volts

2.2 Component Selection and Mechanical Design

The intelligent logistics vehicle had its solid structure design in this project done using the SolidWorks software. The compatibility of the performance of the motor, wheel assembly, and motor driver board was the first step. Once the selection and procurement of components had been completed, all the relevant non-self-designed components were reverse modeled. These parts were then taken into SolidWorks to be assembled virtually. The assembly range also covered all the non-self-designed components except the Bluetooth communication module, ultrasonic sensor module, 5.0 V auxiliary power supply battery, and 11.1 V power supply battery. Jointly with the functional needs of the smart logistics vehicle and the real physical characteristics of the non-self-designed parts, the structural design and 3D

development of the vehicle frame was done with several sets of iterative optimization being adopted in the design process.

2.2.1 Driving mode and wheel set

Motor - The JGB37-520 series permanent magnet brushed Hall encoder DC gear motor will be used with an intelligent logistics transportation application in this project; this has a rated voltage of 12 V (Fig. 1a). The essential benefits of the product are torque output, preciseness in speed measurement, and compatibility in the construction. Our motor has a weight of 152 g, and this satisfies the weight criterion that is lightweight. It is constructed of high strength aluminum alloy with the combination of heat dissipation capabilities and mechanical strength. It has a small structure, a high gear meshing accuracy, low noise of working, and long service life when it has a gearbox diameter of 37 mm. The motor has four ratios, which relate to each other in a linear form, whereby, the higher the gear ratio, the more the torque, and the lower the rotational speed. Together with the requirements of the design speed of the project, this is the 1:90 model that is chosen. In this model, no-load rotational speed is lowered to 110 rpm, the rated rotational speed is lowered to 85rpm, the stall torque is increased considerably to 15 kg·cm and the rated torque is 2.6 kg·cm. Power output and motion stability can be brought into equilibrium through its rotational speed and stall torque. The motor contains a rated current of 0.36 A and a rated power of 7-9 W, which can be used with an 11.1 V lithium battery power supply. Output shaft is a 6mm diameter, D-shaped, eccentric shaft which has high torsion resistance and connection integrity.

The motor incorporates an AB two-phase Hall encoder, fitted with a 11-wire powerful magnetic code disc, outputting the ordinary quadrature square wave signals. The direction of rotation can be identified by phase difference between A and B phase pulses, the pulse frequency provides a feedback on the rotation speed, and the real time feedback on rotational speed and displacement is possible. The encoder is an operating voltage of 3.3 V-5 V, 3.3 V PH2.0-6PIN interface, and with an inbuilt pull-up and shaping circuit. It can be directly linked to I/O ports of the mbed LPC1768 main controller making integration of circuits easier. The motor has the technology of quadruple frequency that enhances the accuracy of measurements by 4 times through the capture of the edge transition signal of the two phases which gives the vehicle a guarantee of accurate movement.

The motor is rigidly attached to the pre-determined screw holes on the motor bracket using six screws of M4-size. The mounting holes are also structured based on the arrangement of the fixing holes on the motor base ensuring that the perpendicularity error of the motor and the vehicle chassis after installation is not more than 1.0, in order to prevent the vibration and noise generated by the lack of coaxiality when using it. In the course of installation, the diagonal tightening method is used to screw and the torque is carefully managed to avoid the concentration of the stress. The D-shaped eccentric shaft of the motor faces the outer side of the chassis, and the axes of the four motor shafts lie in the same horizontal plane, with the spacing between the axes consistent with the track width.

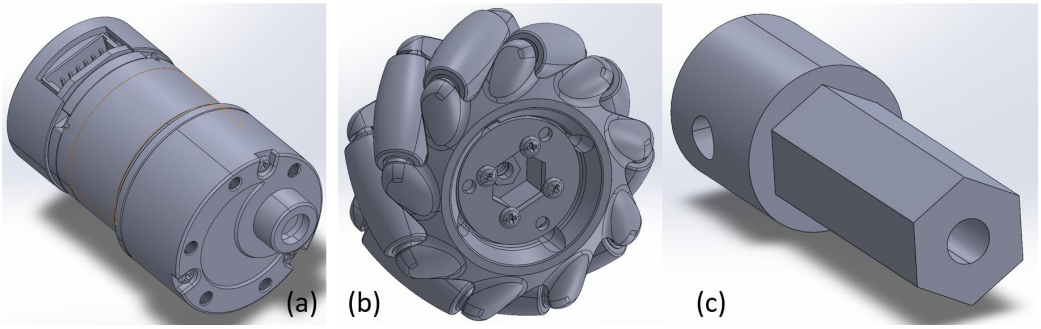

Figure 1. The geometric modeling based on physical object of: (a) JGB37-520 series DC gear motor, (b) Mecanum wheel and (c) Connecting shaft between the motor and wheel.

Wheel base - Standard-specification Mecanum wheels are selected as the motion execution components for this project. Each hub has a single weight of 120 g, featuring high rigidity and the ability to withstand a radial load exceeding 5 kg, which fully meets the project requirements. The wheel has a diameter of 65 mm and a width of 30 mm. The wheel surface is made of wear-resistant rubber, and the hub is made of polyurethane, which is fully compatible with the omnidirectional movement and load-bearing requirements of the vehicle (Fig. 1b).

The fundamental benefit of its capability is in the omnidirection capability of movement. The surface is irregularly spaced with rotating diagonal rollers that are free to rotate creating an angle of 45 degrees with the wheel axle. With the difference in speed as well as steering combinations of the four sets of wheels forward motions, back motions, sideways motions, and circular motions in place are attainable without the requirement of an extra steering system. It fits places with low dimensions like warehouse and workshop, which raises operational adaptability significantly and enhances the efficiency of using space. Regarding performance, the 65 mm diameter in the ground makes the movement stable and

the 30 mm on the wheel width allows a larger contact area of the ground (unit pressure of 0.2 MPa at least), which not only increases the adhesion with the ground, but also prevents scratching onto the ground. The rubber wheel surface and polyurethane rollers are very elastic and resistant to wear as well as have long lifetime.

The wheel is designed in a standard way and there is a hexagonal hole reserved within the hub to align with the connecting shaft. Maintenance and replacement is easier through the universal interface. The other end of the connecting shaft is worked to a D-shaped opening of the same 6mm D-type eccentric shaft of motor. Once axially sleeved onto the motor output shaft, one M4 set screw is used to lock it to the radial threaded hole of the connecting shaft to provide both axial and circumferential fixation and eliminate slippage in transmission of power. The other end is turned into a normal cross section of a hexangle (exactly the same size as the hexangle slot in the Mecanum wheel hub) (Fig. 1c). When it is axially inserted into the wheel body it is secured with an M4 bolt using the hole that has been reserved at the wheel end and at the end of the connecting shaft using the threaded hole to ensure that the wheel body does not move axially. The 4 wheel sets, which are assembled with motors, connecting shafts, and wheels are attached to the bottom of the bottom acrylic plate by hole-to-hole connection using M4 screws through the motor brackets.

2.2.2 Basic frame structure and electronic component layout

To meet the lightweight and visualization requirements, the vehicle frame adopts a basic structure composed of double-layer transparent acrylic (PLEXIGLAS, a kind of PMMA) sheets, studs, and screws. Two 3 mm-thick acrylic sheets are laser-cut according to the design drawings, and M4 screws are locked together with studs in pairs to form a rigid double-layer vehicle body. All connecting holes are reserved in accordance with the M4 screw standard, with a hole position tolerance of ±0.1 mm. At the bending positions, the radius is set to be no less than the material thickness to prevent cracking. For ease of assembly, during the design of the acrylic plates, both plates are provided with large-area square hollowing-out at the center, facilitating the "internal wiring" of power cables and other connections to avoid interference with hardware components. The specific design drawings of the acrylic plates are shown in the following figure (Fig. 2).

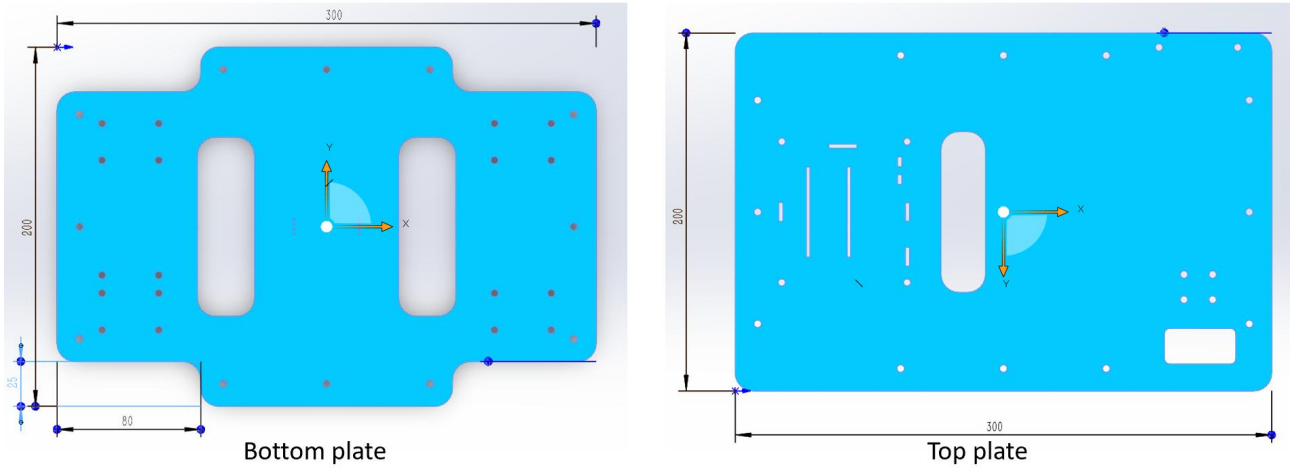

Figure 2. The specific geometry design sketch of the bottom plate and top plate.

The two plate have a difference in design. In the case of the first plate (bottom plate), the processing of wheel avoidance is done at the four corners to minimize the width of the track to increase the rigidity and compactness of the entire vehicle. As the connecting medium between the motors and the sheet, the first plate is reserved with mounting holes for the motor brackets. In the second plate (top plate), hollowing-out processing needs to be carried out at the rear right side of the plate in order to fit the drive rear gear of the tipping bucket, and dedicated mounting holes also are cut at the tipping bucket drive motor.

Regarding the selection of placement positions for the motor driver board and PCB board in modeling, the motor driver board is fixed to the geometric center of the upper surface of the bottom plate (Fig. 3a). Being located at the central position, the driver board enables the interfaces of the four drive motors to form equidistant and shortest wiring paths with the corresponding pin headers of the driver board, thereby preventing wire wear and open circuits caused by excessive bending.

The PCB is attached on the upper surface of the top acrylic plate with the PCB mounted at the front side so that the PCB and lower driver board edge to edge gives an edge-to-edge staggered distribution in the vertical projection to prevent electromagnetic coupling between the electronic devices of the two plates (Fig. 3b). The PCB has the ultrasonic sensor and Bluetooth module and their layout also addresses the objectives associated with sensor operating conditions

and Signal Integrity (SI). As the ultrasonic ranging sensor should not have its field of view blocked at the moment of operation, the PCB layout can avoid mechanical interference of ultrasonic path, whereas the Bluetooth communication module should be situated away of intensive sources of electromagnetic interference like the motors and driver board on the bottom layer so that the remote control signals can be transmitted stably.

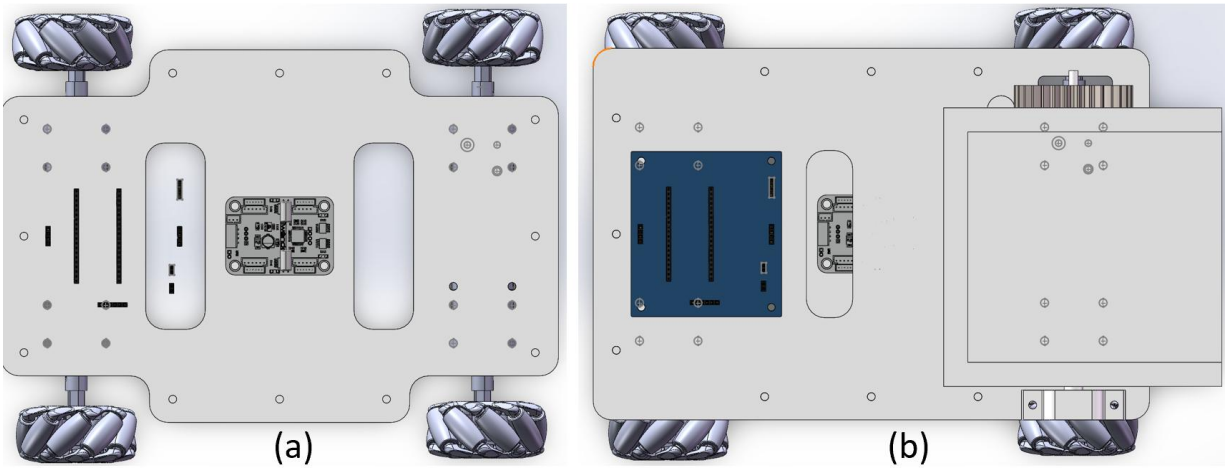

Figure 3. The placements of: (a) Motor driver board and (b) PCB board.

2.2.3 Unloading mechanism

The unloading mechanism of this project adopts a "single-side gear-driven tipping bucket structure" (Fig. 4a). The design objectives are stable unloading under a 2.0 kg rated load, high transmission efficiency, and easy maintenance. Hydraulic rod drive scheme was eventually abandoned partly because of its mass, great volatility, and inability to be shrunk down. The unloading system is completely built inside the back side of the upper plate of the vehicle, which is of acrylic material; there are three main parts that are used in the unloading mechanism: the tipping bucket body, the gear transmission system is a single side, and the drive motor. The tipping bucket body is integrally formed of ABS engineering plastic, with external dimensions of 150 mm×150 mm×50 mm and inner tank dimensions of 137 mm×124 mm×40 mm.

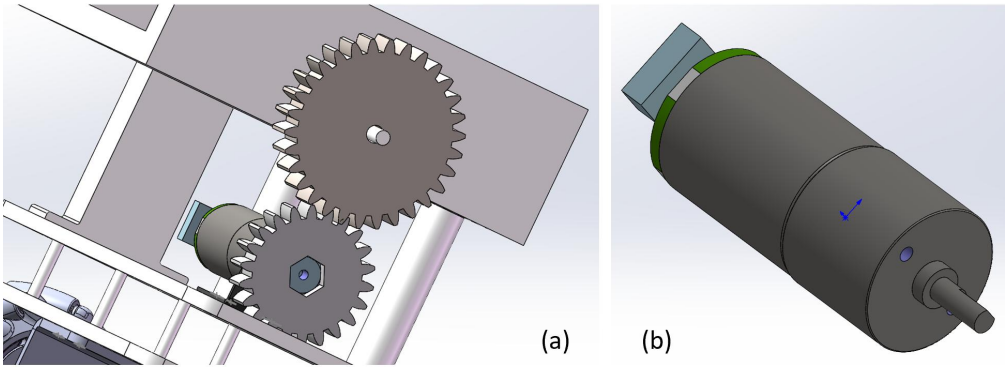

Figure 4. The single-side gear-driven tipping bucket structure: (a) The driving side and (b) MG370 DC gear motor.

Combined with the load requirements of the tipping bucket and the characteristics of gear transmission, the MG370 DC gear motor is selected (Fig. 4b). With a weight of only 95.5 g and a compact structure, it is suitable for the installation space of single-side transmission. It has a rated voltage of 12 V, which is compatible with the vehicle's 11.1 V lithium battery power supply system. The reduction ratio is 1:34, the rated speed is 300 rpm, the stall torque is 1.5 kg·cm, and the rated torque is 8.7 kg·cm. After gear transmission, it can meet the torque requirement for lifting the tipping bucket. The motor is integrated with an AB two-phase Hall encoder (13ppr). The encoder has an operating voltage range of 3.3-5 V and an integrated pull-up and shaping function, which can be directly connected to MCU I/O ports without the need for additional signal processing modules. The encoder outputs standard quadrature square waves. The motor's rotation direction can be determined through the phase difference between the A and B phase pulses, and the pulse frequency is proportional to the rotational speed, supporting closed-loop control of speed and rotation angle. It can be combined with a PID control algorithm to optimize control accuracy. Based on the motor's working characteristic of realizing commutation by changing the voltage polarity, PWM signals can be used to precisely adjust the lifting speed and reset action of the tipping bucket, avoiding load impact.

The gearing system is the two-spur gears transmission system. Taking into account the total form and performance of the services it is ultimately concluded that the driving gear is 22 T and the driven one is 30 T, which achieves the amplification of the motor torque by a factor of 1.36. The module and pressure angle is then uniformly set to 2.0 and 14.5 degrees respectively. The correct tooth and gear ratio not only results in the gears having a small footprint but does not compromise on motion accuracy, as well as maximizes the motor torque without becoming overly limitative of speed (the original characteristic of the selected motor is high torque and low speed). Having a driving motor is similar to the wheel set, in that the difference gear is now attached to the drive motor shaft, with a connecting shaft to assure that no relative sliding is happening with the gear. The driven gear is mounted on the vertical central axis on the right side of the tipping bucket, the tipping bucket is then undertaking the circular movement around the center of the driven gear on the time of loading and unloading. The gear that turns is directly rigidly attached to the bowl of tipping and does not move relative to the point of contact.

The working principle is concise and clear. Operators control the unloading and resetting processes of the tipping bucket via Bluetooth. The tipping bucket has a maximum design rotation angle of 90 degrees, but there is significant redundancy in this angle under actual working conditions (mostly 30-45 degrees). After a Bluetooth remote control sends an unloading command, the driving wheel rotates clockwise, and the driven gear drives the tipping bucket to tilt backward for unloading, with the angle freely controllable. After a reset command is sent, the driving wheel rotates counterclockwise, and the tipping bucket returns to the horizontal position. A limiter is installed to prevent excessive resetting of the tipping bucket.

2.3 Final Design Diagram in SolidWorks

The figure below shows the final 3D design model (Fig. 5), which will serve as a reference for assembling the physical cart.

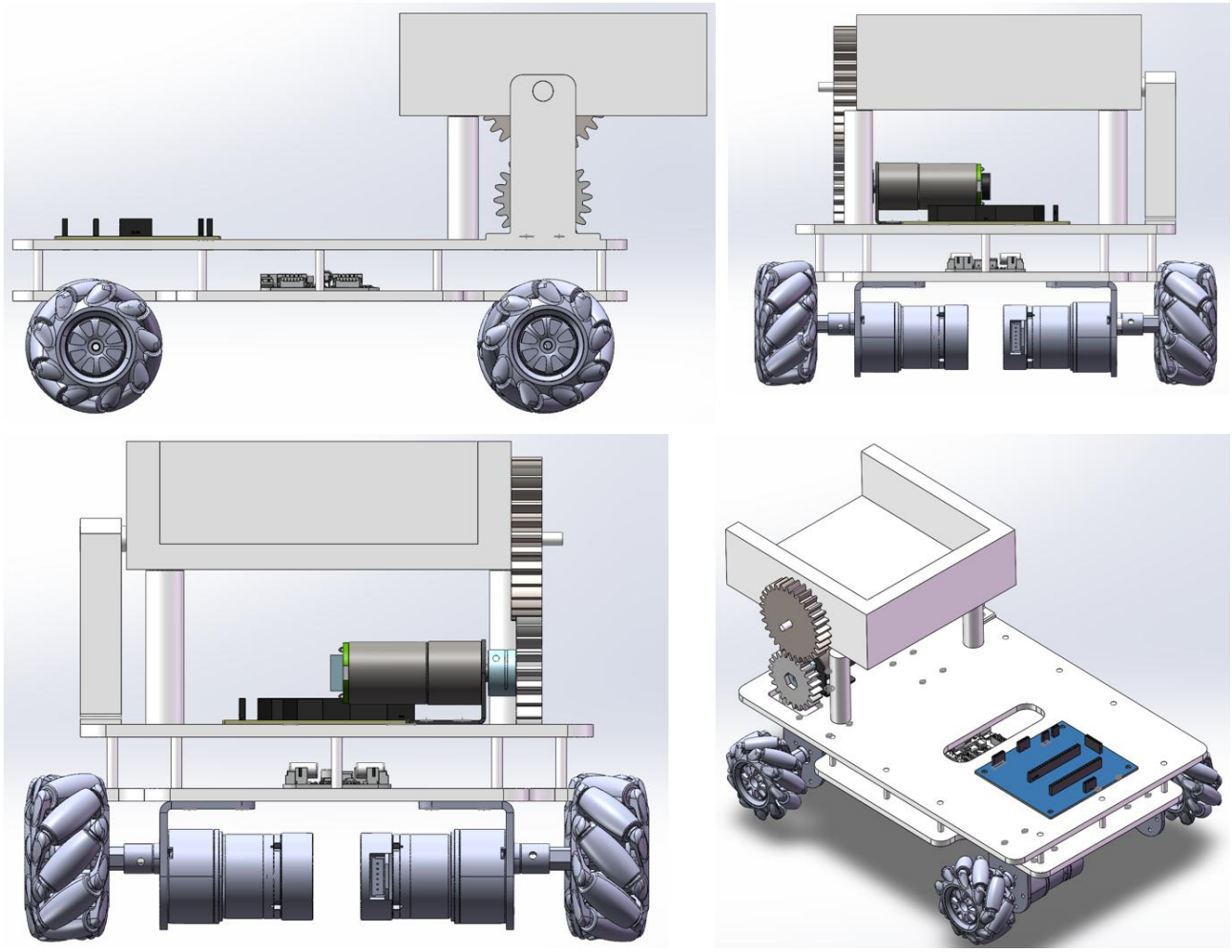

Figure 5. The final design of the geometry model of the vehicle.

3 Mechanical Assembly and Operation Analyse

3.1 ANSYS Mechanical Analysis (Loading Condition)

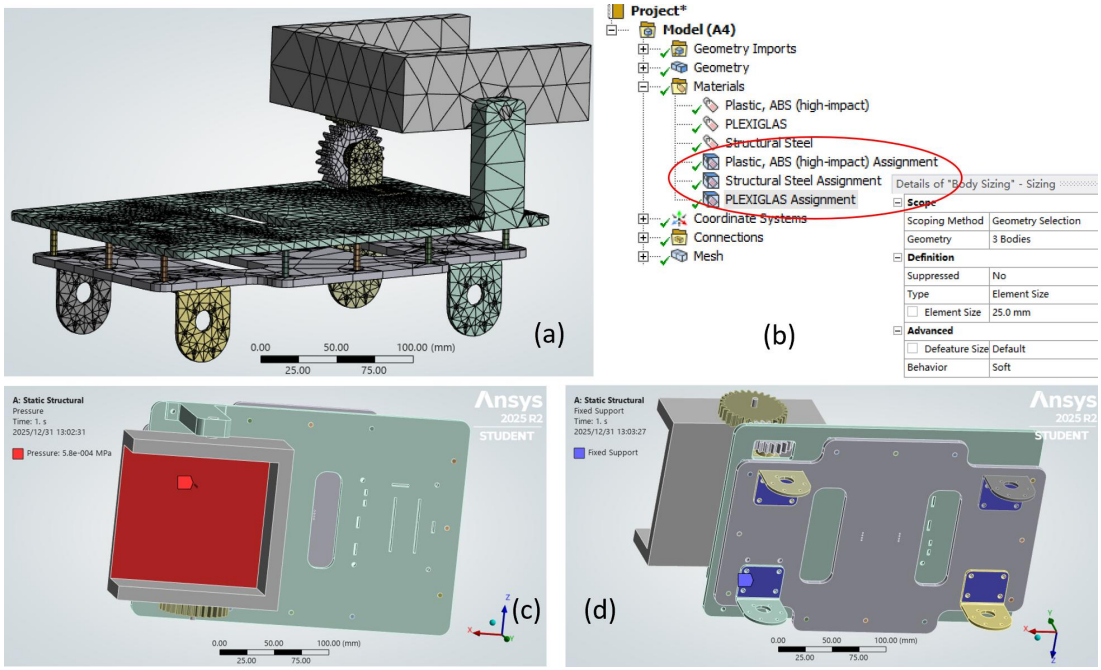

Figure 6. The geometry and settings in ANSYS Workbench: (a) Frame geometry modeling for analysis, (b) Material and mesh settings, (c) Load applied in the tipper and (d) Fixed support setting at the bottom of the motor brackets.

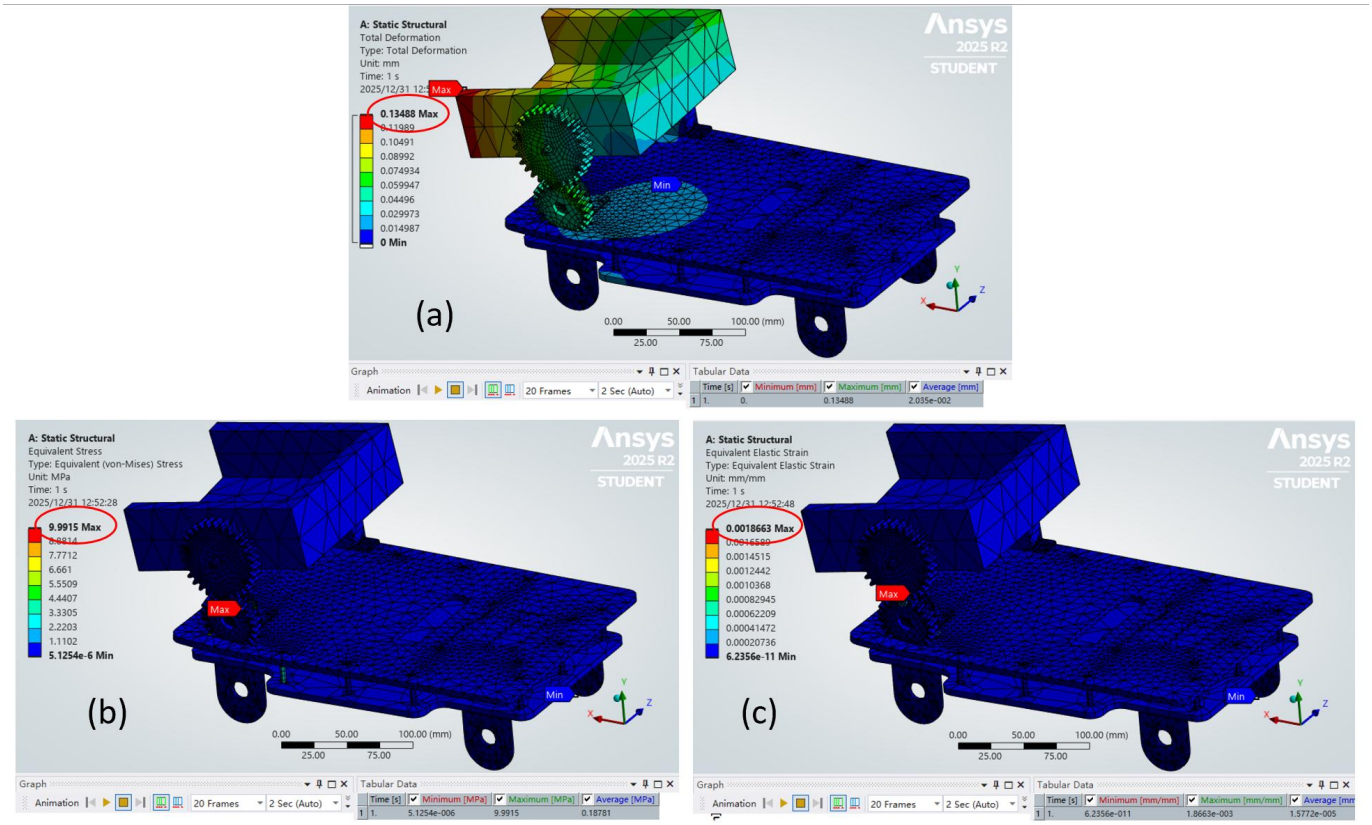

Figure 7. The simulation results under the load of 2kg (1154Pa): (a) The total deformation, (b) The equivalent stress and (c) the equivalent strain.

Prior to the assembly of physical components, to clarify the structural mechanical performance of the vehicle under different load conditions and determine the safe limiting load (if the 2.0 kg preset rated load is feasible or not), multiple

sets of static structural simulation analyses were conducted using ANSYS Workbench. The simulation geometric model was modified and imported based on the SolidWorks design drawings of the entire vehicle: motors, wheels, limiter and electronic components were removed, retaining only the critical frame components (load-bearing structure), and non-critical fine features were simplified to improve computational efficiency. The mechanical parameters of core components were set in accordance with actual material selections.

For material specifications: the tipper and tipper bracket are made of ABS plastic (elastic modulus: 2.4 GPa, yield strength: 44 MPa); the double-plate acrylic plates of the frame are made of PLEXIGLAS (elastic modulus: 3.2 GPa); the studs and four motor brackets are made of stainless steel (elastic modulus: 200 GPa, yield strength: 250 MPa) (Fig. 6a). All material data were retrieved from the ANSYS Engineering Data database. Tetrahedral elements were used for meshing, with a global element size of 25 mm. The average qualification rate of mesh quality was ≥90%, and no warnings or errors were reported by the system (Fig. 6b). Three load cases were configured for the simulation: uniform loads of 1 kg (580 Pa), 2 kg (1154 Pa), and 3 kg (1733 Pa) were applied inside the tipper (Fig. 6c). Constraint conditions were consistent with the actual support state: a Fixed Support was set at the bottom of the motor bracket to restrict horizontal and vertical displacements (Fig. 6d).

Taking the finally determined limiting load of 2 kg as an example, three simulation results are optimistic. The maximum total deformation is 0.13 mm, occurring at the rear edge of the tipper bucket, with a negligible magnitude that does not affect structural functionality (Fig. 7a). The maximum equivalent (Von Mises) stress of the entire vehicle is 9.99 MPa, concentrated at the connecting lugs between the tipper and the frame, and the stress distribution in other regions is uniform, with values below 8 MPa (Fig. 7b). The equivalent elastic strain is extremely small and remains within the elastic deformation range (Fig. 7c).

The multi-load results in Table 2 were compared and structural stress and deformation were found to increase directly with the load. Hence, the 2 kg can be considered as the maximum load of the tipper of the vehicle, and structural strength and stiffness are in the design requirements under this condition. Out of the three output results, the total deformation and maximum equivalent strain may be excluded because their values are much lower than the mechanical property limits of the materials. The tensile yield strength of ABS plastic is 44.0 Mpa and was calculated as the critical Von Mises strength and the safety factor is about 4.40 which shows that there is no potential risk in material failure. Also, the tipper is at the back of the vehicle and therefore despite satisfying the heavy-load requirements in material strength, the Calibrated load upper limit is lower to the extent of avoiding an overturning even during loading and unloading activities- based on the consideration of center-of-gravity.

Table 2. Summary of the figures under three loads in ANSYS solution.

Load	Property	Maximum	Average
1.0 kg (580 Pa)	Von Mises stress (σ_V, MPa)	5.62	9.44×10^{-2}
	Von Mises strain (ε_V)	9.38×10^{-4}	7.93×10^{-6}
	Total deformation (u_{TM}, mm)	6.78×10^{-2}	1.02×10^{-2}
	Safety Factor (SF)	7.83	
2.0 kg (1154 Pa)	Von Misesstress (σ_V, MPa)	9.99	0.19
	Von Mises strain (ε_V)	1.87×10^{-3}	1.58×10^{-5}
	Total deformation (u_{TM}, mm)	0.13	2.04×10^{-2}
	Safety Factor (SF)	4.40	
3.0 kg (1733 Pa)	Von Mises stress (σ_V, MPa)	15.00	0.28
	Von Mises strain (ε_V)	2.80×10^{-3}	2.37×10^{-5}
	Total deformation (u_{TM}, mm)	0.20	3.06×10^{-2}
	Safety Factor (SF)	2.93	

3.2 Bill of Materials and Assembly

The list of materials required for this project is shown in Table 3. This list included components that were 3D printed by the team, designed PCBs and ordered components. The assembly process was divided into three stages: bottom plate assembly, top plate assembly, and tipping bucket mechanism assembly.

Table 3. Bill of materials for the entire project.

Part	Material	Num	Dimension	Specification
Mecanum wheel	Rubber and polyurethane	4	Φ65mm×30mm （Φ×W）	
520 DC gear motor	Metal components	4	152g	JGB37-520
Bottom plate	PLEXIGLAS (PMMA)	1	300mm × 200mm × 3mm（L×W×T）	
Hexagon screw	Stainless steel	24	M4×6	
Pan-head screw	Stainless steel	47	M4×8	
Wheel fixing bracket	Stainless steel	4		37-type Reinforced
PCB and switch		1, 2	8.7mm×8.1mm (L×W)	
double-ended internally threaded cylindrical pin	304 stainless steel	12	Φ6×25×M4	
Top plate	PLEXIGLAS (PMMA)	1	300mm × 200mm × 3mm（L×W×T）	
Four-channel Motor Driver Module		1		Voltage Range: DC 3V-12V
Support column	ABS engineering plastic	2	Φ15mm×60mm (Φ×T)	
Shaft support bracket	ABS engineering plastic	1	32mm×16mm×80.36mm(L×W×T)	
Tipping bucket	ABS engineering plastic	1	external: 150 mm×150 mm×50 mm, inner tank: 137 mm×124 mm×40mm	
Driving gear	PLEXIGLAS (PMMA)	1	22 teeth, Module=2.0, Pressure angle=14.5°	
Driven gear	PLEXIGLAS (PMMA)	1	30 teeth, Module=2.0, Pressure angle=14.5°	
Motor bracket	Stainless steel	1		
MG370 DC Motor	Mental components	1		MG370
Mbed LPC1768	Electronic components	1		LPC1768
TB6612FNG Dual DC Motor Driver Module	Electronic components	1	50mm×50mm (L×W)	TB6612FNG
Pan-head machine screw	Stainless steel	5	M3×6	
Hexagon Head Bolt	Stainless steel	4	M3	
Hexagon Head Bolt	Stainless steel	16	M4	
Hexagonal Coupling	Iron	1	M4 (Inner)	
Coupling	Iron	4	M6(Inner)	
Lithium Battery	lithium components	1	69×55.5×19mm (L×W×T), 142g	11.1V ,2000mAh
Power Bank	lithium components	1	90×60×14mm (L×W×T),100g	5000mAh
Dupont Wire (with a breadboard)	Copper and PVC	-		
Bluetooth module	Mental components	1	30mm×20mm (L×W)	HC-08
Ultrasonic sensor	Mental components	1	40×20×15mm (L×W×T)	HC-SR04

Bottom Plate Assembly - The bottom plate assembly involved the installation of the drive mechanism, the bottom plate and the power source (Fig. 8a). To assemble the drive mechanism, the 520 DC gear motors were first fixed to the wheel fixing brackets, then the couplings were attached to the motor shafts. Finally, the couplings were inserted into the centers of the Mecanum wheels. Four couplings were fixed to the underside of the bottom plate. Additionally, the motor driver module, power bank and lithium battery were placed centrally on the upper surface of the bottom plate. The central placement of the power source and drive board ensured proximity to the various electrical interfaces of the vehicle. This configuration allowed for the efficient arrangement of the drive system components, while also leaving enough space for the Bluetooth module and ultrasonic sensor on the top plate, which enhanced the overall aesthetics.

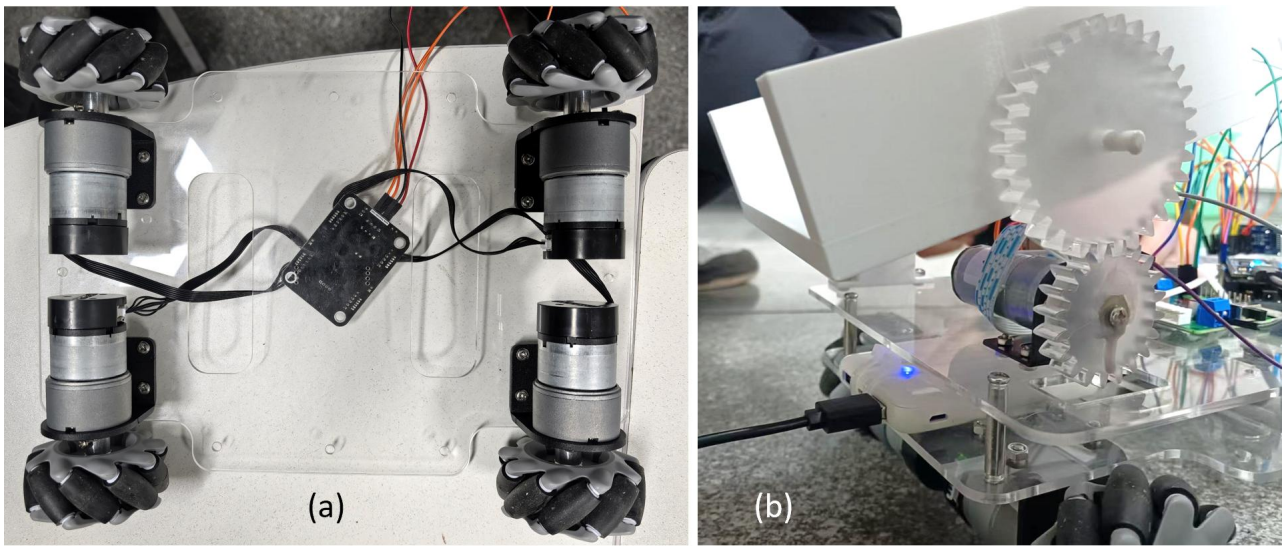

Figure 8. (a) The bottom plate after assembling and (b) The tipping bucket after assembling.

Top Plate Assembly - The top plate was a key structural element that housed the functional components of the vehicle. The front of the top plate was designed with specific cutouts to securely hold the PCB. After fixing the PCB, the Bluetooth module and ultrasonic sensor were connected to the headers of the PCB. The ultrasonic sensor was positioned directly in front of the PCB, aligned with the front of the vehicle, which ensured optimal performance for distance measurement. The Bluetooth module was placed on the left side of the PCB for effective signal reception. The mbed LPC1768 was inserted into the center header of the PCB, enabling easy access for button reading during code execution. Finally, the top plate and bottom plate were connected using pins, providing sufficient height clearance between the two boards.

Tipping Bucket Mechanism Assembly - The tipping bucket mechanism was mounted at the rear of the top plate. The assembly began with the attachment of the shaft support bracket to the left rear of the top plate using screws. The gear motor, motor bracket and hexagonal coupling were connected in sequence. The driving gear was then attached to the hexagonal coupling and connected with the TB6612FNG Dual DC Motor Driver Module, forming the gear drive mechanism (Fig. 8b). The driven gear was installed on the right-side axle of the tipping bucket, while the left axle of the tipping bucket was inserted into the hole at the top of the shaft support bracket. This completed the assembly of the tipping bucket mechanism, with the driving and driven gears meshing.

Wiring and Connections - The PCB was connected to the driver board and voltage regulator board using Dupont wires. The lithium battery supplied power to the driver board, which acted as the motor of the four-wheel motors whereas the power bank supplied power to the voltage regulator board, which served as the motor of the gear motor. The Dupont wires were passed, through the lengthy cutout on the center of the top plate, and then connected to the components on the bottom plate, so that an inside will provide clean and secure wiring, but without interfering with the wheels when they are on the move.

3.3 Physical Structure Testing and Mobility

The Figure 9 illustrates the physical model that is assembled and that which will act as the design reference of the real vehicle. The project that is eventually to be delivered will receive small modifications as will be discussed subsequently in the text. The system was taken to test on different terrains. The drive mechanism was also stable with the bottom plate and the car could move as usual under a manual push of 2 kg weight. The motor-driver contact was rigid, and the

screws were excellent fixers of the driving mechanism, the Mecanum wheels were rotated at the same axis as the motor shafts, and the motion was guaranteed.

The bottom plate and top plate provided excellent support for the tipping bucket mechanism, power sources and electrical components. The pin connections between the two plates were robust, when under minor shaking or light impacts, there was no loosening of the screws, nor was there any misalignment of the plates. This confirmed that the pin connection design effectively distributed the load and restricted relative movement between the two plates. The power source and drive board also remained securely in place between the two plates, which demonstrated the reliability of the pin connections.

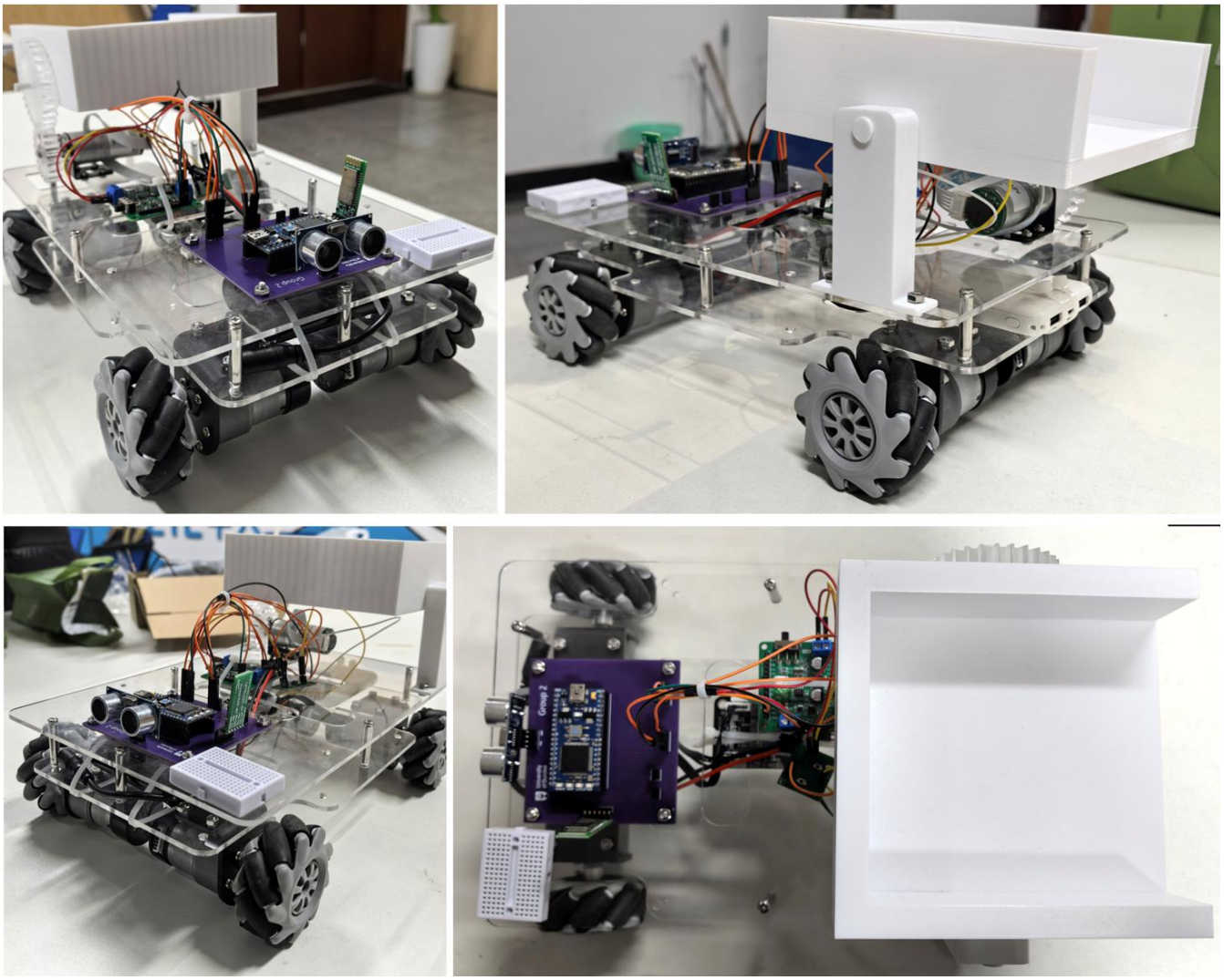

Figure 9. The assembled physical model as the design reference.

The tipping bucket mechanism effectively supported a 2 kg load. Even when the load was placed at the far end of the bucket, the bucket did not shift significantly downwards, also there was no risk of the bucket flipping over. During tipping, transmission was stable and there was no tooth skipping between the driving and driven gears. There was no overturning of the bucket during the unloading process. The two limit columns were determined to be removed because even if the tipping bucket over-resets, it can be manually leveled, thus preventing damage to parts such as the drive gear and shaft caused by the mechanism jamming forcefully.

Both the Bluetooth module and the ultrasonic sensor were securely connected to the PCB via headers, and the Dupont wire connections were stable. The overall system demonstrated excellent stability and reliability, with no issues during operation.

4 Microcontroller Implementation

4.1 Embedded System Design Based on Mbed Microcontroller with Extension

To implement the functional modules of the logistics robot, the project establishes a preliminary embedded system based on the mbed LPC1768 microcontroller and its effective integration with the components mentioned in the hardware section (including drivers, motors, ultrasonic sensors, Bluetooth modules, etc.). The microcontroller features abundant peripheral interfaces. In this project, interfaces and communication protocols including IIC, PWM, UART, and GPIO are employed to construct a sophisticated sensing-decision-making-control closed-loop system. The diagram below illustrates the interfaces utilized and their interconnections in this project (Fig. 10).

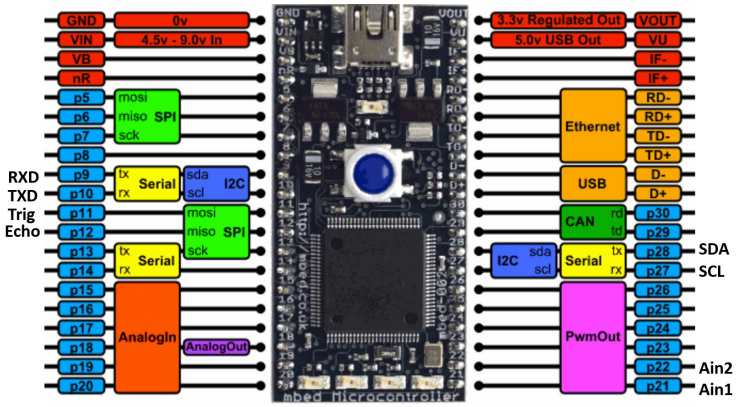

Figure 10. The interfaces utilized and interconnections in mbed LPC1768 microcontroller.

Specifically, pins p9 and p10 are allocated to the Bluetooth module, pins p11 and p12 to the ultrasonic module, pins p28 and p27 to the IIC interface of the chassis, and pins p21 and p22 to the PWM inputs of the auxiliary motor. Consequently, four subsystems have been developed: A chassis motion control system via the IIC bus (data transmission); An auxiliary loading/unloading mechanism controlled through PWM interfaces (control signal generation); Bluetooth communication via the UART serial port (command reception) and Ultrasonic sensor data acquisition through GPIO pins (data collection).

4.2 IIC Communication Protocol and Chassis Motion Control System

The control of chassis motors represents the most complex and critical component of this project. To conserve GPIO pin resources of the microcontroller and achieve independent four-wheel drive, this project employs a control scheme based on the Inter-Integrated Circuit (IIC) bus.

IIC is a multi-master, multi-slave serial communication bus that enables data transmission between devices using only two signal lines: the Serial Data Line (SDA, corresponding to pin p28 on the development board) and the Serial Clock Line (SCL, corresponding to pin p27). The physical layer adopts an open-drain output structure, where the master device (LPC1768) and slave devices (motor driver boards) can only pull the bus low to logic 0, but cannot actively drive it high to logic 1. Therefore, pull-up resistors must be externally connected to the bus to maintain a high-level state during idle periods. This design not only mitigates the risk of power supply short circuits but also implements wired-AND logic, providing the hardware foundation for the IIC bus arbitration mechanism. Data transmission timing follows the standard master-slave communication protocol: when SCL is high, the master pulls SDA low to generate a start condition, signaling the beginning of transmission and bus occupation; subsequently, the master transmits a 7-bit slave device address followed by a read/write bit (write operations are primarily used in this project, with R/W=0); during the 9th clock cycle, if the driver board successfully receives the address, it pulls SDA low to send an acknowledgment (ACK) signal, and failure to receive an ACK (e.g., when the driver board is unpowered) will result in initialization failure in the code; data is transmitted in 8-bit bytes with the most significant bit (MSB) first, and the receiver sends an ACK after each byte transmission; upon completion of transmission, the master pulls SDA high while SCL remains high to generate a stop condition and release the bus.

In terms of hardware integration, the LPC1768 establishes communication with the PCA9685 chip on the driver board via the aforementioned IIC pins. The PCA9685 is a 16-channel, 12-bit resolution IIC interface PWM driver that functions as a key control signal expander in the system. Instead of directly generating PWM signals required for motor driving, the LPC1768 encapsulates speed and direction commands into IIC data packets and transmits them to the PCA9685. The chip then outputs multiple channels of high-precision PWM waveforms and logic level signals, which drive the four chassis motors through H-bridge circuits to achieve omnidirectional control. This configuration enables precise multi-motor control using only 2 IIC pins.

For software implementation, the project encapsulates IIC communication details within the DCMotor class, with the core control logic implemented in the setLR function. This function first maps speed commands in the range of -100 to 100 to register values between 0 and 4095 corresponding to 12-bit resolution. Control signals are then generated based on the sign of the speed value: for positive speeds, the corresponding PCA9685 channels are configured to output high levels (AIN1/BIN1) and low levels (AIN2/BIN2) on respective pins, with a square wave of corresponding duty cycle output on the PWM channel; for negative speeds, the high-low level logic is inverted to achieve reverse motor operation. Finally, the IIC.write() function from the mbed OS is called to package and transmit register addresses and calculated data to the PCA9685. This implementation significantly conserves GPIO resources of the LPC1768, enabling smooth speed regulation and differential steering of four motors using only 2 pins.

4.3 PWM Signal Generation and Auxiliary Motor Control

Unlike the indirect IIC (Inter-Integrated Circuit) control adopted for the chassis, the auxiliary motors (configured for the loading and unloading mechanism) implement direct PWM (Pulse Width Modulation) control via the LPC1768 microcontroller. PWM is a technical method that utilizes digital output to simulate analog signals. By switching digital pins at an extremely high frequency, the average voltage of the output signal can be precisely regulated. Two core academic concepts related to PWM require clarification for a comprehensive understanding of this project. First, duty cycle refers to the proportion of time that a signal remains at a high level within a single pulse period, expressed by the formula $D = \frac{T_{on}}{T_{period}}$. Next, frequency is defined as the number of times a signal repeats per unit time (per second).

Table 4. PWM control logic diagram table.

Operating State	p21 (PWM)	p22 (PWM)	Motor Behavior
Forward Rotation (Unloading)	PWM (Duty Cycle: 12%)	0 (GND)	Current Flow: A -> B
Reverse Rotation (Retraction)	0 (GND)	PWM (Duty Cycle: 12%)	Current Flow: B -> A
Stop	0	0	Free Stop

A frequency of 50Hz is selected in this project, which is determined based on the inductive characteristics of the motor. Excessively high frequency will lead to increased switching losses of the H-bridge driver, while excessively low frequency may induce vibration or noise in the motor. A 20 ms period (corresponding to 50 Hz) is a classic and widely accepted parameter for driving brushed DC motors. The motor responsible for driving the rotation of the tipping bucket employs a high-performance motor driver module based on the Toshiba TB6612FNG chip. This module adopts a MOSFET H-bridge architecture, integrating MOSFET-based H-bridge circuits. This configuration features extremely low on-resistance (Ron), which significantly reduces switching losses and heat generation — a characteristic that is crucial for the unloading function of the trolley.

In the code implementation, the initial duty cycle is set within the range of 0.01 (1%) to 0.12 (12%). This parameter setting is specifically designed to prevent damage to the mechanical structure caused by excessive motor rotational speed.

4.4 UART Serial Communication and Bluetooth Human-Machine Interaction

UART Principle - The LPC1768 microcontroller establishes a communication link with the HC-08 Bluetooth module via the Universal Asynchronous Receiver-Transmitter (UART) protocol, so as to enable the functions of wireless remote control (Mode 9) and command transmission (Mode 15). As an asynchronous serial communication protocol, UART's core advantage lies in that the communicating parties do not need to share a clock signal; instead, they achieve synchronous data transmission through a predefined baud rate. In this project, the baud rate is set to 9600bps, which means 9600 bits of data can be transmitted per second. Meanwhile, the communication data frame format is configured as follows: 1 low-level start bit, 8 data bits (for transmitting ASCII characters), 1 high-level stop bit, and no parity bit—this configuration simplifies the communication process while ensuring data transmission efficiency.

Hardware Cross-Communication - Hardware connections adhere to the "cross-connection" principle:

LPC1768 TX (p9) -> HC-08 RXD: The microcontroller sends data (e.g., distance information feedback) to the mobile application.

LPC1768 RX (p10) <- HC-08 TXD: The microcontroller receives control commands from the mobile application.

Data Processing Logic - The system uses a serial port class to encapsulate the UART data processing process, and continuously monitors the status of the receive buffer via a polling mechanism in an independent thread. When character data is detected, the system parses the command type based on ASCII codes and executes corresponding motor control operations (e.g., forward movement, stop). At the same time, the system transmits the current motor operating status back to the control terminal in real time, establishing a full-duplex human-machine interaction closed loop to ensure the real-time performance and reliability of command execution and status feedback.

4.5 GPIO Control and Ultrasonic Data Acquisition

Environmental perception serves as the technical foundation for realizing the functions of automatic obstacle avoidance (Mode 9) and intelligent following (Mode 15). In this system, the HC-SR04 ultrasonic module is employed to conduct distance data acquisition.

The distance measurement process is based on the Time-of-Flight (ToF) principle, which calculates the target distance by detecting the reflection time of sound waves. The specific workflow is as follows: the LPC1768 microcontroller pulls the Trig pin (p11) high for at least 10 microseconds, triggering the sensor to transmit 8 ultrasonic pulses at 40kHz. When the sensor detects the reflected sound waves, it sets the Echo pin (p12) to a high level, and the duration of this high level is equal to the round-trip propagation time of the sound waves.

At the software implementation level, the system uses a Timer object to measure the high-level pulse width of the Echo pin. To avoid program deadlock caused by abnormal conditions such as disconnected sensors, a timeout counter is integrated into the while loop. The standard physical formula adopted for distance calculation is presented below.

$$\text{Distance (cm)} = \frac{\Delta t\ (\mu s) \times 0.034}{2} = \frac{\Delta t}{58}$$

Data conversion is performed by considering the round-trip propagation characteristic of sound waves, based on the speed of sound in air (approximately 340 m/s or 0.034 cm/μs). To address resource contention issues in a multi-threaded environment, the system implements mutex protection before operating the GPIO pins, and executes an unlock operation after the measurement is completed. This ensures the timing integrity of the Trig and Echo signals, thereby guaranteeing the accuracy and reliability of distance data acquisition.

4.6 Summary of the Microcontroller Utilization

In summary, the mbed LPC1768 microcontroller has successfully integrated a complex electromechanical system through the flexible adoption of multiple communication protocols and control interfaces. Control signals generate precise digital packets via the IIC protocol to control the chassis, while PWM analog signals are employed to regulate auxiliary motors. For data acquisition, high-precision GPIO timing is utilized to capture environmental depth information. The human-machine interface (HMI) achieves real-time wireless command stream processing through the UART protocol.

This LPC1768-based hardware implementation scheme not only fully leverages the chip's on-chip resources but also significantly reduces wiring complexity through rational protocol selection—such as using IIC instead of GPIO for multi-motor control. This optimization effectively enhances both the reliability and response speed of the overall system.

5 Electrical Circuit and PCB Design

5.1 Circuit Connection Idea

5.1.1 Power supply logic and unified ground

Power supply - The core electronic components of this project include 5 DC motors, 1 Bluetooth communication module, 1 ultrasonic ranging sensor, 1 mbed LPC1768 main control board, 1 four-channel motor driver board and 1 two-channel motor driver board. The power supply requirements of each component present a dichotomous distribution: DC motors and the motor driver board have a rated operating voltage of 12 V, belonging to high-power power-level power supply requirements; while the Hall encoders integrated in DC motors, Bluetooth communication module, ultrasonic ranging sensor, and mbed LPC1768 main control board have a rated operating voltage of 5 V, belonging to low-power signal-level power supply requirements.

To realize the integrated design of the power supply system and simplify the circuit architecture, this project adopts a power supply topology scheme of "single main power supply + DC-DC buck adaptation". The core uses an 11.1 V lithium battery (which can be regarded as equivalent to 12 V) as the main power supply, paired with a high-performance LM2596 series DC-DC buck module to complete voltage conversion. This buck module avoids interference with signal acquisition of low-power sensors and encoders.

Based on the above architecture, the DC power output by the 12 V lithium battery is directly connected to the power input terminal of the motor driver boards. While providing working power for the motor driver boards, it supplies the rated 12 V driving voltage to five DC motors through the power output interface of the driver board, ensuring the movement and unloading functions of the vehicle. The 12 V auxiliary output port reserved on the motor driver board outputs 12V voltage for the LM2596 buck module. After voltage stabilization and filtering by the buck module, it outputs stable 5 V DC power, which is respectively supplied to the Bluetooth communication module, ultrasonic ranging sensor, and Hall encoders integrated in each motor, so as to ensure the stability of the communication link and ranging signals.

Regarding the power supply design of the mbed LPC1768 main control board, initially the Vin terminal of mbed LPC1768 was used as the main power supply, from which unknown faults arose. After repeated trials and error analysis during the research process, it was concluded that the essence of the frequent logic errors at its Vin power input interface is the electromagnetic interference and voltage drop in the power-level power supply circuit coupling to the signal-level circuit. Hence, a 5 V (2.1 A) regulated power bank is used to independently power the mbed board through a standard USB Type-A power supply interface (Fig. 11a). This design can effectively cut off the transmission path of electromagnetic interference and voltage ripple from the main power supply circuit. Meanwhile, the built-in Lithium Battery Management System (BMS) of the power bank realizes overvoltage, overcurrent, and short-circuit protection, further improving the robustness of the power supply system.

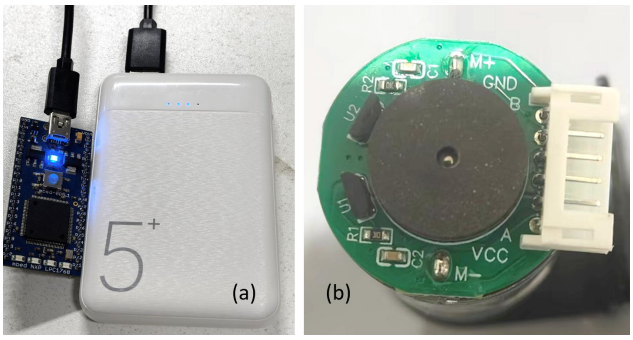

Figure 11. (a) The power bank used to independently power the mbed board through a Type-A power supply interface and (b) The DC motor with six interfaces arranged (both for tipping bucket and Mecanum wheels).

Unified ground - Prior to the formal PCB board design, DuPont wires were used for wiring testing to better verify the correctness of the wiring logic. To achieve ground normalization, a simple breadboard was adopted, where all ground wires are uniformly connected to form a dedicated centralized grounding point, serving as an artificially designated zero potential point (GND) .

From the perspective of Electromagnetic Compatibility (EMC) design, the single-point grounding scheme can effectively eliminate the ground potential difference between modules caused by distributed grounding. Ground potential difference is a key source of crosstalk between modules, and the mutual interference between high-frequency modules and sensitive circuits can be significantly attenuated through this scheme. Meanwhile, the grounding path is simplified and the ground impedance is reduced, which ensures the consistent potential of the ground pins of each module. This is conducive to the stable transmission of analog signals (such as encoder speed feedback signals) and digital signals (such as Bluetooth communication signals), enhancing the anti-interference capability of the entire system. The successful operation of the component preliminary experiment provides a reliable basis for the subsequent design of the PCB grounding system and the overall wiring scheme.

5.1.2 Analysis of single component wiring (pin)

Motor - As shown in Figure 11b, the DC motor is equipped with six interfaces arranged from top to bottom, namely M+, GND, B, A, VCC, and M-, where M+ and M- are the main power terminals designed to connect to an external 6-12 V power supply to provide the required electrical energy for the motor's operation, and the remaining four interfaces (GND, B, A, 5V) are dedicated to the normal operation of the encoder—5 V and GND form a stable power supply loop for the encoder, while the A and B interfaces are used for signal transmission and detection, and by measuring the

voltage between the A-phase/B-phase output of the encoder and the encoder's negative terminal (GND), the angular position and rotational speed of the motor rotor can be accurately calculated through pulse signal analysis.

Motor driver board - In this project, the DC motors serve two application scenarios, namely wheel drive and tipping bucket device drive. The four motors used for wheel drive are integrated into the circuit system through a dedicated four-channel driver board, a centralized connection method that optimizes the layout of power lines and signal lines, enhances the stability of the drive system, and facilitates unified control of the four wheels (e.g., speed synchronization, direction coordination) by the main controller. The motor used for the tipping bucket device uses another two-channel driver board which can also realizes its connection to the circuit system.

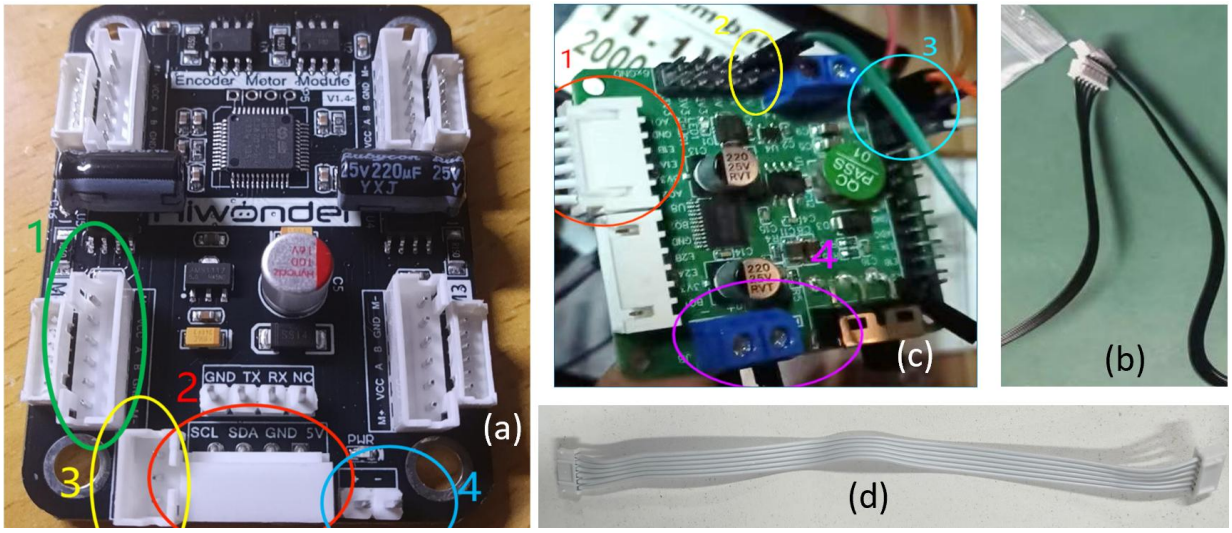

Figure 12. (a) The pin headers and interfaces on driver board of the four DC motors and (b) its driver board wires. (c) The pin headers and interfaces on driver board of tipping bucket and (d) its driver board wires.

The four DC motors controlling the rotation of the Mecanum wheels will be controlled by a dedicated driver board (Fig. 12a). By connecting the corresponding interfaces on the DC motors to the corresponding pin headers of the driver board (Fig. 12a, Point 1) via wires (Fig. 12b), all four motors will be integrated into this driver board, and the relevant signals can be connected to the mbed board through the IIC output interface (Fig. 12a, Point 2), among which SCL is connected to pin 27 of the mbed board and SDA to pin 28 of the mbed board (the relevant principle has been mentioned earlier). To ensure the high-level state of SCL and SDA, it is necessary to connect these two ports to the Vout of the mbed using pull-up resistors. However, preliminary experiments revealed that pin 28 and pin 29 of the mbed appear to be equipped with inbuilt pull-up resistors, so no external resistors were added. Nevertheless, to mitigate the uncertainty of the mbed's internal pull-up resistors, additional considerations are incorporated into the PCB design (see below). The 5 V port can be used when the mbed board supplies power to the driver board, but since the driver board requires a 12 V input for normal operation, this port will be left floating, and the GND is normally connected to the unified ground point. In addition, the driver board is also equipped with a power input terminal and a power output terminal. In this project, the pin header on the left is used for 12 V input (Fig. 12a, Point 3) and is connected to the positive and negative terminals of the 12 V battery, while the pin header on the right is used for additional voltage output, which can output 12 V from the driver board (Fig. 12a, Point 4) and will be connected to the LM2596 buck module to generate 5 V for other components. The two-channel drive module for the tipping bucket assembly is as illustrated in the diagram above(Fig. 12c). Location 1 is connected to the six terminals of the motor via the wires presented in Fig. 2d. Point 2 is designated for accessing the output voltage of the buck converter module, along with a common ground wire. Point 3 serves as the signal reception interface: the Ain 1 interface is connected to pin 21, while the Ain 2 interface is connected to pin 22; both interfaces are used to enable the driver board to receive the PWM output from the mbed controller. Point 4 is configured for an external battery to supply 12 V power to the motor driver.

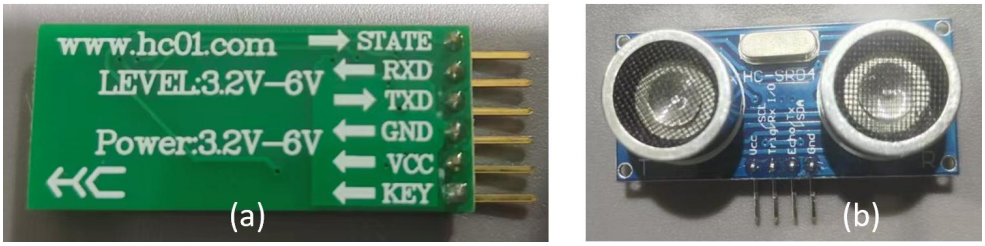

Figure 13. The wiring configuration of: (a) Bluetooth communication module and (b) Ultrasonic ranging sensor.

Bluetooth communication module - Figure 13a illustrates the wiring configuration of the Bluetooth module, which consists of six pins arranged sequentially from top to bottom. Among these, four pins (RXD, TXD, GND, and VCC) are utilized in this project. VCC is designated for 5V voltage input, so it is connected to the output terminal of the buck module. TXD functions as the data transmission terminal, while RXD serves as the data reception terminal. To enable communication, the output terminal of the Bluetooth module must be connected to the input terminal of the mbed, and the input terminal of the Bluetooth module must be connected to the output terminal of the mbed. In this project, the TXD terminal of the mbed is configured to pin 9, and its RXD terminal is configured to pin 10. Accordingly, the TXD terminal of the Bluetooth module should be connected to pin 10 of the mbed board, and the RXD terminal of the Bluetooth module should be connected to pin 9 of the mbed board. As for GND, it is uniformly connected to the common ground line.

Ultrasonic ranging sensor - The wiring of the ultrasonic sensor is shown in the Figure 13b. It has four pins from left to right, namely VCC, TRIG, ECHO, and GND. Among them, the VCC pin, as mentioned above, is connected to the output terminal of the voltage reduction module. In this project, pin11 is used as the Digital out port and pin 12 as the Digital in port, so the TRIG terminal will be connected to pin 11 of the mbed board, and the ECHO terminal will be connected to pin 12 of the mbed board. The TRIG pin is defined as an active-high trigger signal input terminal, responsible for receiving the digital control signal output by the mbed LPC1768 microcontrolle. The microcontroller outputs a high-level pulse through the pin 11 port to trigger the sensor to transmit ultrasonic waves. The ECHO pin serves as an echo signal output terminal, which can convert the ultrasonic propagation time into a corresponding high-level pulse signal and feed it back to the microcontroller for signal processing. The duration of the high-level ECHO signal has a linear relationship with the distance from the sensor to the target object; the microcontroller captures the pulse width of the ECHO signal through the timer interrupt function, thereby achieving distance calculation. As mentioned above, the GND terminal will be connected to the unified ground connection point.

5.2 PCB Schematic

After conducting relevant tests to verify the correctness of the circuit wiring, the PCB design was initiated. The most frequently used electronic component in this PCB design is the female pin header. Two rows of 2×20 female headers are employed for mounting the mbed board. Each 2×20 female header is interconnected with a wire to simulate the layout configuration of a standard mbed expansion board. This design ensures that even if malfunctions occur in the internal wiring of the PCB design or subsequent adjustments are required, temporary connections can be established using external Dupont wires, thereby avoiding delays to the experimental progress. As mentioned earlier, if the internal pull-up resistors of the mbed malfunction, these redundant female pin headers can be used to add pull-up resistors of any resistance value externally to adapt to the IIC signals.

During the use of Dupont wires, it was found that a large number of wires were required, so to simplify the circuit, it is essential to reduce the use of wires, and female pin headers were adopted to minimize unnecessary wiring—such as the connections between the four interfaces of the Ultrasonic ranging sensor and the power supply as well as the unified ground point. Taking the Ultrasonic ranging sensor as an example, it has four interfaces that need to be connected to the circuit, therefore a 1×4 female pin header is used to insert the four pins of the Bluetooth module. After the female pin header is soldered onto the PCB board, these pins can be connected to the corresponding parts that need to be linked through the copper traces on the board.

Meanwhile, since there are too many components requiring 5 V power supply, the power supply needs multiple wires for voltage division, so a 1×2 female pin header is arranged at the bottom left corner of the schematic diagram for inputting the 5 V voltage from the output terminal of the buck module, and this voltage will be connected to the positive electrodes of various components through the copper traces on the board to realize power supply. In addition, to prevent potential future needs for additional unified ground connections, an extra 1×4 female pin header is added in the schematic diagram for connecting other potential ground wires.

The other connection logic has been covered in the preceding section and will not be repeated here. The figure above is the schematic diagram of the PCB design (Fig. 14).

Subsequently, the design of the PCB layout was initiated. To facilitate the physical installation and normal operation of the actual product, the position of each component was carefully considered. The ultrasonic sensor was placed at the center of the topmost part of the PCB, corresponding to the central position at the front of the overall vehicle. Meanwhile, considering the position of the driver board, the power inlet and the female headers for connecting IIC ports on the PCB were arranged at the bottom of the PCB, which is exactly aligned with the opening position on the second layer of acrylic board directly above the driver board. In addition, taking into account that the tipping bucket is installed at the rear of the vehicle and its motor is located on the right side of the rear, the corresponding female header for connecting the tipping bucket motor was also arranged at the bottom right corner of the PCB. To facilitate the physical

connection of the PCB in practical applications, specific identification marks are printed on the top-layer silkscreen. These include annotations for various interfaces of the mbed board and different ports of the Bluetooth module.

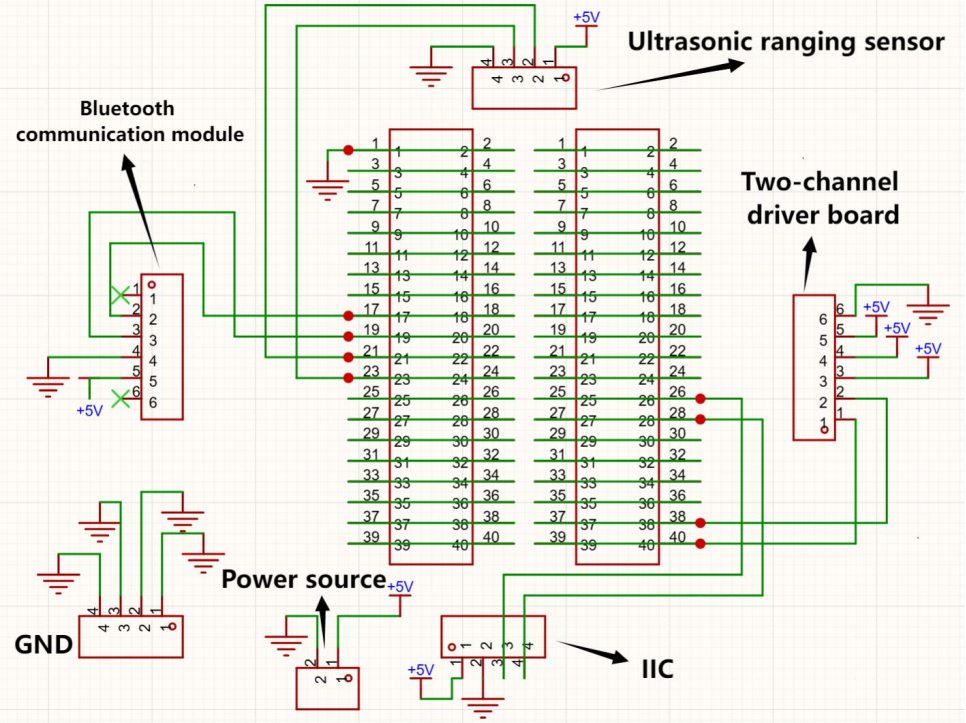

Figure 14. The schematic diagram of the PCB design.

The project adopts a basic two-layer PCB, which is divided into a top layer and a bottom layer. In the design, it was intended to separate the functions of the top layer and the bottom layer to facilitate error detection and circuit inspection. At the same time, to avoid signal conflicts between the bottom and top layers. The bottom layer was designed exclusively for power supply, while the bottom layer is mainly used for wiring related to signal transmission. In the design of the bottom power layer, considering that the traces may carry large currents, all power supply lines on the bottom layer are designed with a width of 40 mil, which differs from the 10 mil width adopted for other signal traces. This configuration is specifically optimized to enhance heat dissipation performance. On the top layer, some wiring inevitably intersects, such as the intersection between the wire connecting pin 27 to SCL and the wire connecting pin 28 to SDA. Therefore, to avoid conflicts, via was used at the impending intersection to route the wires from the bottom layer to the top layer, and then route them back to the bottom layer after the intersections. In order to comply with the safety rules for PCB traces, all right-angle turns should be avoided. Therefore, in the design, all obvious turning points were deliberately set with 45-degree bends to achieve this.

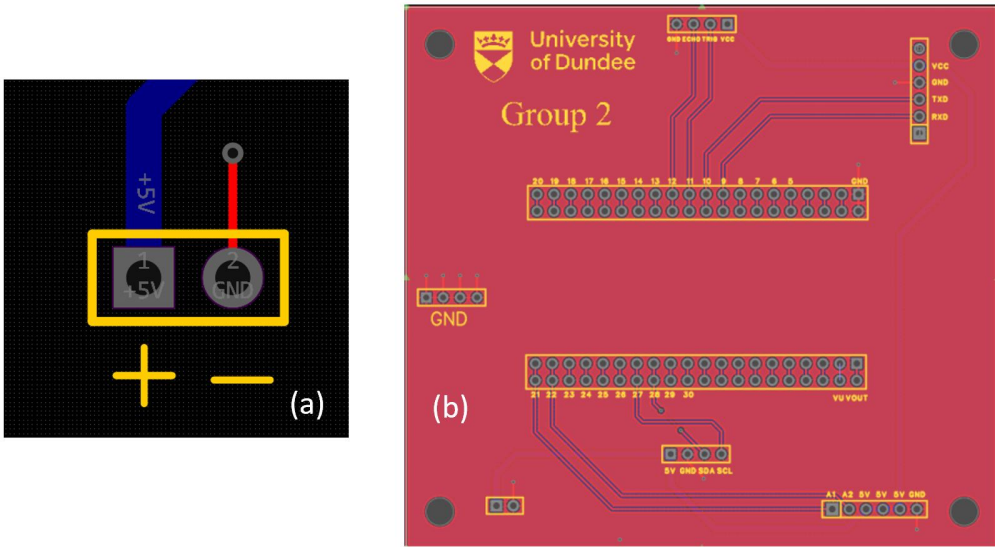

Figure 15. (a) The via placed adjacent to each ground pad and (b)The copper pouring process.

After all wiring has been configured, copper pouring operations are implemented in the PCB design. This process not only unifies the ground plane but also significantly improves the heat dissipation efficiency of the PCB. Additionally, a via is placed adjacent to each ground pad and electrically connected to it (Fig. 15a). Copper pouring is then performed separately on both the top and bottom layers, with the via enabling electrical continuity between the ground planes of the two layers (Fig. 15b). This measure further reduces ground impedance and enhances the stability of the grounding system. Subsequent to the copper pouring, check whether there is any issue of excessive proximity between the ground wires and other traces, and separately inspect the spacing between all traces on both the top surface and the bottom surface of the PCB. The final PCB layout is shown in Figure 16a.

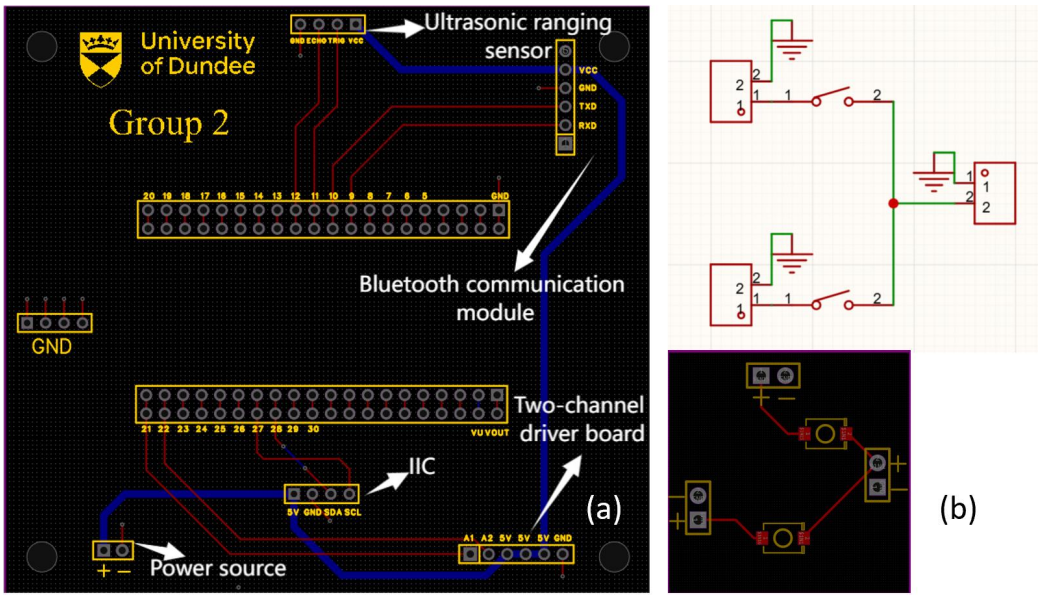

Figure 16. (a) The final PCB layout and (b) A printed circuit board (PCB) for controlling the battery switch.

Since each test requires the plugging and unplugging operations of the 12 V battery's positive terminal connection, which is excessively cumbersome, a printed circuit board (PCB) for controlling the battery switch has been designed (Fig. 16b). In the figure, the positive and negative terminals of the 12 V battery are to be connected to the 1×2 female header on the right end, which is linked to two single-pole single-throw (SPST) switches and thereby connected to the other two 1×2 female headers. These two female headers are used to supply power to the driver board and the motor of the hopper device respectively. This PCB enables the free switching on and off of the battery power, which can improve the efficiency of the experimental process while facilitating the use of the vehicle.

5.3 PCB Physical Model

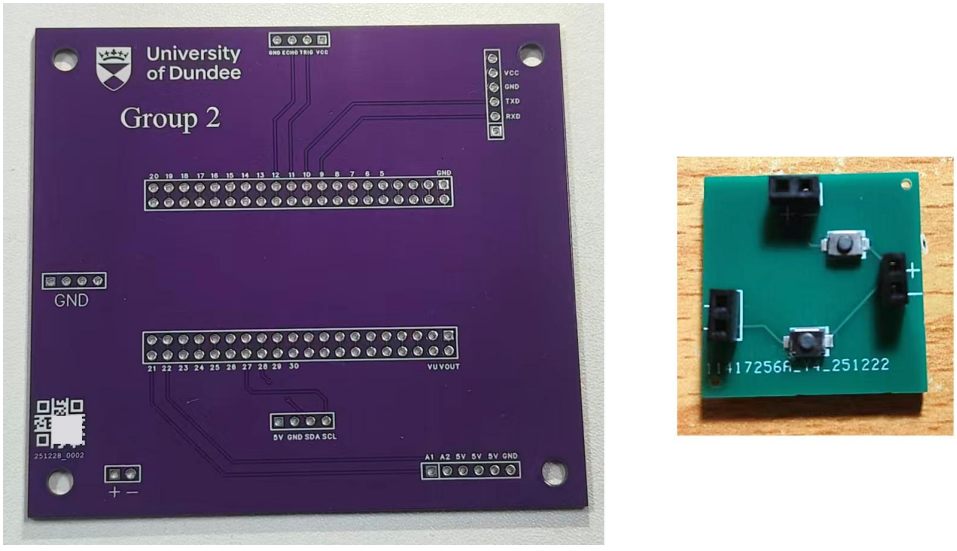

Figure 17. The final physical images of two PCB boards.

Figure 17 illustrates the final physical images of two PCB boards, which were manufactured and deployed in the project. By this stage, all wiring issues for core components have been resolved, with the PCB and battery switch control board for the mechanical structure now completed. It achieves efficient hardware integration with anti-interference capabilities as well as establishes a robust hardware foundation for subsequent software development and functional implementation.

6 Software Development and Function Realization

6.1 Code Design Philosophy (Modular Architecture)

In view of the functional implementation of the logistics trolley (such as forward/backward movement, left/right turning, tipping bucket unloading, ultrasonic obstacle avoidance, etc.; see subsequent sections for details), the hardware components mentioned earlier (motor driver board, drive motor, tipping bucket motor, etc.), the mechanical structure design characteristics (e.g., the frame formed by double-layer acrylic matched with lock studs, the tipping bucket driven by a single-side gear), and the implementation and application of the microcontroller, the software development will be conducted through three links: logical framework construction, code writing, testing and implementation. To ensure the stability and debuggability of the control system, a modular and iterative development strategy is adopted for these three links, and multi-threading features are leveraged to handle concurrent tasks.

6.2 Modular Description (Detailed Development of Each Mode)

The software component is separated to various modes whereby each mode shall correspond to an autonomous testing or multifunction integrated module. The design allows the isolation of hardware fault in the early development phase, testing hardware motors, sensors and communication modules individually and finally, good modules are merged into the advanced automated efforts. The modes are structured in an iterative process: begin with simple tests (e.g. forward rotation, reverse rotation, acceleration and deceleration of single motor), continue with implementation individual functions (e.g. Bluetooth and ultrasonic sensing) and then further to overall integration of the functions and expansion of functions (e.g. sensor fusion to avoid obstacles when running the trolley, path reuse, and behind the vehicle). This strategy gives the project a more conceptualized structure and achieves a more multifaceted scope of functions. This style of design, where various operating modes are chosen depending on the variables and altered behavior of the trolley depending on inputs of the inputs in the specific mode is known as a Finite State Machine (FSM) [6]. Finite State Machine (FSM) is a mathematical system that is used to model object behavior. FSM has been extensively used in embedded system software to control the complicated logic of the system. Its conceptualization is: the system may be in a single state at any given moment; as the particular input (Input/Event) is received, the system leaves the current state and goes into another state which causes the system to do the particular action (Action).

6.2.1 Two-Layer Finite State Machine with Limited Modules

The first layer: System-Level Operational Mode Selection. At the entrance of the main() procedure, there is a designed motion controller (controlled by the global MODE variable) in the form of a state assessor taken as a macroscopic state management system. It is designed to achieve modular testing, the working principle is the following: with the system turned on, it booted into a particular functional mode, based on the supplied setting (e.g., Mode 1 to test a motor hardware, Mode 9 to test a remote control functionality). The design allows the developers to control failures, test each hardware part of the system individually (IIC bus, Bluetooth, sensor) and finally bring them together to form the complex Mode 15.

The second layer: Task-Level Dynamic FSM (Mode 15). Within the core functional Mode 15 (Smart Logistics), a standard dynamic finite state machine is implemented to handle complex logical interactions, which is a key highlight and crucial part of the software design of this project. An enumeration type, enum State { IDLE, AUTO_FOLLOW, RECORDING, REPLAYING, RETURNING }, is defined here to manage the following state transitions:

(1) IDLE (Idle State): The default state of the system, waiting for user commands. At this time, the motors are stopped, and the system is in a low-power standby or manual fine-tuning mode.

(2) RECORDING (Recording State): Upon receiving the Bluetooth command '3', the system performs real-time monitoring of the user's remote control operations and records each action along with its duration (Timestamp) into a memory array.

(3) AUTO_FOLLOW (Auto-Follow State): Upon receiving the Bluetooth command '1', the system takes over the chassis control right and automatically adjusts the vehicle speed based on the ultrasonic distance using a PID or hysteresis algorithm.

(4) RETURNING (Returning State): Upon receiving the Bluetooth command '5', which is the state with the most complex algorithm. The system reads the recorded data, executes the "Inverse Kinematics" algorithm, and automatically navigates the trolley back to the starting point.

The rationale for adopting a state machine is self-evident. The behavior of the system at any moment is predictable, thereby avoiding logical confusion (e.g., preventing the false triggering of manual control during auto-returning). If the "auto-patrol" function needs to be added in the future, it is only necessary to introduce a new PATROL state in the enum and define the corresponding transition conditions, without rewriting the entire codebase—this greatly enhances scalability. Meanwhile, its debuggability is also improved: by printing the current state via the serial port (e.g., [M15] State: RECORDING, [M15] State: IDLE), logical errors can be quickly located.

6.2.2 Software Development Evolution

The software development followed a path from simplicity to complexity, undergoing three primary phases.

Hardware Verification Phase (Mode 1-8): This phase focused on debugging underlying drivers. For instance, Mode 1 exclusively tested the forward and reverse rotation of a single-channel PWM motor to validate the H-bridge drive circuit, while Mode 8 verified data transmission and reception of the Bluetooth module to ensure unobstructed communication links. This phase established the basic functionalities of the DCMotor library and AuxControl class.

Functional Integration Phase (Mode 10-14): During this phase, sensor data was integrated with motor control. For example, Mode 13 enabled chassis control via the PC serial port and introduced ultrasonic ranging data for the first time to dynamically adjust motor output in real time. Key technical challenges were addressed in this phase, including interference issues arising from the coexistence of the IIC bus and PWM signals. The system's robustness was significantly enhanced through the implementation of power-on delay and dynamic memory allocation mechanism.

System Application Phase (Mode 9 and 15): This represents the final deliverable form of the software. Mode 9 integrated remote control with safe obstacle avoidance, while Mode 15 implemented complex intelligent logistics tasks such as autonomous following and path memory. These two modes collectively embody the final achievements of the software development process.

Regarding the Core Functional Logic, the software finally delivered in this project mainly consists of the following two core modes.

Mode 9: Remote Control with Obstacle Avoidance. This mode adopts a dual-thread architecture. The main thread is responsible for parsing Bluetooth commands, controlling the omnidirectional movement of the chassis, and the loading/unloading actions of the auxiliary motor (forward rotation/reverse rotation/jog operation). The background logic thread runs at high priority, monitoring the ultrasonic distance in real time. Once an obstacle within 20 cm ahead is detected, the system will immediately suspend the remote-control commands and enforce the obstacle avoidance sequence of "emergency stop - backward movement - steering", which effectively prevents collisions caused by human operation errors.

Mode 15: Smart Logistics and Path Memory. This mode manages the behaviours via a Finite State Machine (FSM).

IDLE (Idle State): Waits for user commands.

AUTO_FOLLOW (Follow State): Based on ultrasonic ranging data, it maintains a distance of 20–30 cm from the target object through a simple Hysteresis Control algorithm.

RECORDING (Recording State): The system monitors the user’s manual operations in real time, and records each action command (e.g., forward movement, unloading) and its corresponding duration into a memory array.

RETURNING (Return State): This is the innovative feature of this project. The system reads the path data in the memory in reverse order and generates "reverse commands" (e.g., converting "forward movement" to "backward movement"), thereby realizing automatic reverse movement back to the starting point, which simulates the homing process of a logistics vehicle after completing delivery.

6.2.3 Detailed Explanation of 15 Modes

In accordance with the overall testing and debugging workflow, a total of fifteen modes have been developed, which are enumerated as follows.

6.2.3.1 Debug Modes (Mode 1-8, 10-12, 14)

Mode 1 (Single Motor Control Mode): This mode is designed for independent testing of the left chassis motor (M1). Sending the character "f" via the serial port commands M1 to rotate forward at speed level 50; sending "b" commands it to rotate backward at speed level 50; and sending any other character causes M1 to stop immediately.

Mode 2 (Automatic Square Movement Mode): No manual commands are required. All motors rotate forward at speed level 50 for 5 seconds and then stop, with the trolley remaining stationary and ceasing to respond to subsequent commands.

Mode 3 (Speed Acceleration Test Mode): No manual commands are required. The motor speed starts from 0 and increases by 5 speed units every 500 milliseconds until reaching the upper limit of 80 speed units, after which it maintains constant-speed operation.

Mode 4 (Independent Front and Rear Wheel Control Mode): Sending the character "f" via the serial port commands the front wheel motors (M1, M3) to rotate forward at speed level 50 while the rear wheels stop; sending "r" commands the rear wheel motors (M2, M4) to rotate forward at speed level 50 while the front wheels stop.

Mode 5 (Automatic Front-Rear Wheel Switching Mode): No manual commands are required. The front wheels rotate forward at speed level 50 for 3 seconds, then switch to the rear wheels rotating forward at speed level 50 for 3 seconds, and this cycle repeats continuously.

Mode 6 (Automatic Circular Movement Mode): No manual commands are required. All motors rotate forward at speed level 50 for 3 seconds, then the left wheels stop while the right wheels rotate forward at speed level 50 for 2 seconds to perform a clockwise rotation, and this cycle repeats continuously.

Mode 7 (All-Wheel Timed Forward Movement Mode): Sending the character "g" via the serial port causes all motors to rotate forward at speed level 50 and stop automatically after 5 seconds; the trolley remains stationary when no command is sent.

Mode 8 (Bluetooth LED Control Mode): Sending the character "1" via the Bluetooth APP toggles the state of LED1 (on/off); sending any other character leaves the LED state unchanged.

Mode 10 (All-Wheel Forward and Reverse Control Mode): Sending the character "f" via the serial port commands all motors to rotate forward at speed level 50; sending "b" commands them to rotate backward at speed level 50; sending "s" stops all motors immediately.

Mode 12 (Speed Increment Mode): The initial speed is set to 0. Sending the character "a" via the serial port increases the speed by 10 speed units, with continuous transmission of "a" enabling continuous speed increase; sending "s" resets the speed to 0 and stops the motors.

Mode 14 (General-Purpose Reserved Debug Mode): This mode is reserved for debugging expansion functions and dedicated testing of core components to supplement the existing test coverage.

6.2.3.2 Basic Modes (Mode 9, 13)

Mode 9 (Bluetooth Motion Control Mode) initialization: Chassis motors stop and auxiliary motors reset; the Bluetooth serial port implements non-blocking reception, while the PC serial port adopts blocking reception; command processing and distance detection logic are activated.

Core Command Responses: Sending "f" (forward movement, initial speed 20, left and right wheels synchronized), "b" (backward movement, initial speed 20), "a" (accelerate, step size 5, upper limit 80), "d" (decelerate, step size 5, lower limit 0), "l" (turn left, steering value +5, upper limit 60), "r" (turn right, steering value -5, lower limit 60), "z" (direction reset, steering value cleared to zero and "smooth stop"), "s" (stop, speed and steering value cleared to zero immediately); Auxiliary motor control: "t" (forward rotation, initial value 0.12f), "g" (reverse rotation, initial value 0.12f), "v" (stop), "y" (accelerate, step size 0.02f, upper limit 1.0f), "h" (decelerate, step size 0.02f, lower limit 0.1f).

To address the inertial impact issue caused by sudden braking in vehicle dynamics—a phenomenon commonly referred to as "nosedive" in automotive dynamics—the project innovatively integrates a "smooth stop" mechanism into the motion control system. This functionality draws on the technical content of an invention patent developed by team members Jin Li and Shucen Guo, entitled *A kind of "Smooth Stop" Sensor System Integrated into Front Calipers of New Energy Vehicles* (Patent Publication: CN120986359A) [7]. In the above-mentioned patent, accurate measurement of force of the automotive calliper by a composite sensor allows the vehicles to slow down to zero speed with minimum constant acceleration. The braking force through sensor feedback control mechanism is taken to achieve the stable parking like the logic in the patent which is the "smooth stop" mechanism of the project. The devised open-loop linear deceleration algorithm is essentially different compared with emergency braking commands which immediately removed the H-bridge PWM signal. The smooth braking logic, on the contrary, increases the motor duty cycle

progressively over a fixed time period. This algorithm approximates a constant negative acceleration process which enables the differential drive system to pass smoothly between a moving condition and rest. This is especially a vital feature under high transport transportation scenarios as it does not only prevent mechanical straining on the transmission system but also provides the stability of the transported goods. Ultimately, the speed of the motor irrespective of the operating condition attains a linear attenuation ($R=R_0-k\times t$) until the vehicle reaches full stationarity.

Mode 13 (Automatic Obstacle Avoidance Mode) trigger condition: The ultrasonic sensor updates the measured distance every 50 ms; the mode is activated automatically when the obstacle distance is less than 20 cm.

Obstacle Avoidance Actions: Immediately stop the chassis motors (wait 500 ms) → left wheel reverses at speed 50, right wheel rotates forward at speed 50 (600 ms, 45-degree steering) → resume forward movement (speed 40) → restore normal control logic after a 500 ms wait.

6.2.3.3 Intelligent Mode (Mode 15: Intelligent Memory and Following Mode)

(1) Auto-Following Mode (Triggered by Command "1")

Response Logic: The system ignores all commands except "x" and adjusts its operating state solely based on ultrasonic distance measurement results: when the distance is greater than 30cm, the trolley moves forward at speed level 50; when the distance is between 20 cm and 30 cm (20 cm < distance ≤ 30 cm), it moves forward at speed level 30; when the distance is between 15 cm and 20 cm (15 cm ≤ distance ≤ 20 cm), it stops moving; when the distance is between 10 cm and 15 cm (10 cm < distance < 15 cm), it moves backward at speed level 30; when the distance is less than or equal to 10 cm, it moves backward at speed level 40; when the distance exceeds 40 cm, it stops and waits. Exit Mechanism: Sending the command "x" immediately terminates the auto-following mode, stops all movements, and returns the trolley to the initial stationary state.

(2) Path Recording and Playback Mode

Path Recording (Triggered by Command "3"): The system records all subsequent operational actions and their corresponding durations (e.g., "f→3 s", "l→2 s"). Sending the command "s" stops the recording process and saves the complete path data to memory. Forward Playback (Triggered by Command "4"): The system executes all recorded actions in the exact order of their recording, strictly adhering to the original action types and duration parameters.

Reverse Playback (Triggered by Command "5"): The system executes the recorded actions in reverse chronological order, with all directional commands inverted (e.g., the original action "move forward for 3 seconds" is converted to "move backward for 3 seconds"), enabling the trolley to accurately return to its starting position.

Emergency Stop (Triggered by Command "x"): Sending the command "x" at any stage of auto-following, path recording, or path playback immediately halts all ongoing operations of the trolley, restores it to the initial stationary state, and requires a new command to be sent to re-enter the corresponding operational mode.

6.2.4 Logical Conception and Flow Diagram

Prior to the testing and development of each Mode, a systematic process from functional design to logical conception must be conducted, and this process is formally represented using a program flow diagram. The diagram structure adopts a hierarchical framework, starting with an overall System Main Flow and then proceeding to the branch flows corresponding to each Mode. Given the total of 15 Modes, this section focuses on elaborating on the key function-integrated modules, namely Mode 9 (Remote Control and Obstacle Avoidance Module) and Mode 15 (Intelligent Logistics Module). The following figure is the main flow diagram (Fig. 18a).

In the main flow diagram, System Startup (Power On) refers to the power-on process of the microcontroller. Initialization is the most critical step: the program first executes a sleep function to introduce a delay, waiting for power supply stabilization to prevent the IIC bus from hanging. Subsequently, it initializes the chassis motors (via IIC), auxiliary motors (via PWM), ultrasonic sensors, and Bluetooth module in sequence. During Mode Selection, the program reads the value of the volatile integer variable MODE. If MODE is set to 9, the system enters Remote Control Mode; if set to 15, it enters Intelligent Logistics Mode; otherwise, it proceeds to other debugging modes.

Figure 18b shows the workflow of the remote control and obstacle avoidance of Mode 9 (the dual-thread architecture that allows listening to commands and acquiring environmental perceptions at the same time). When Mode 9 is activated, an initialisation process is performed at first and the dual-threaded workflow is initiated; then there is parallel execution of the two independent threads. Thread 1 is a daemon background process. It keeps on updating the ranging data of the ultrasonic sensor in cyclic fashion, real-time ascertains whether the distance to the object before it is less than 20 cm. When this state is met it aggressively takes power and performs the avoidance mechanism of obstacle evasion of stop - turn left - move forward. When the maneuver is complete it takes a 30 ms sleep before going back to the ranging update loop. Thread 2 serves as a foreground interaction process. It waits for Bluetooth command input in a

cyclic manner. Upon receiving a character command, it first identifies the command type: if the command is a character (e.g., "f", "b"), it performs the chassis movement operation; if the command is a character (e.g., "t", "g"), it controls the auxiliary motor to complete the loading/unloading operation; if the command is "k", it executes the fine-tuning maneuver of "rotating 45 degrees". Once it has performed the relevant command, it will go into sleep (10 ms) and go down to the Bluetooth input waiting loop again. This design realizes the functions of safety perception and command interaction under Mode 9.

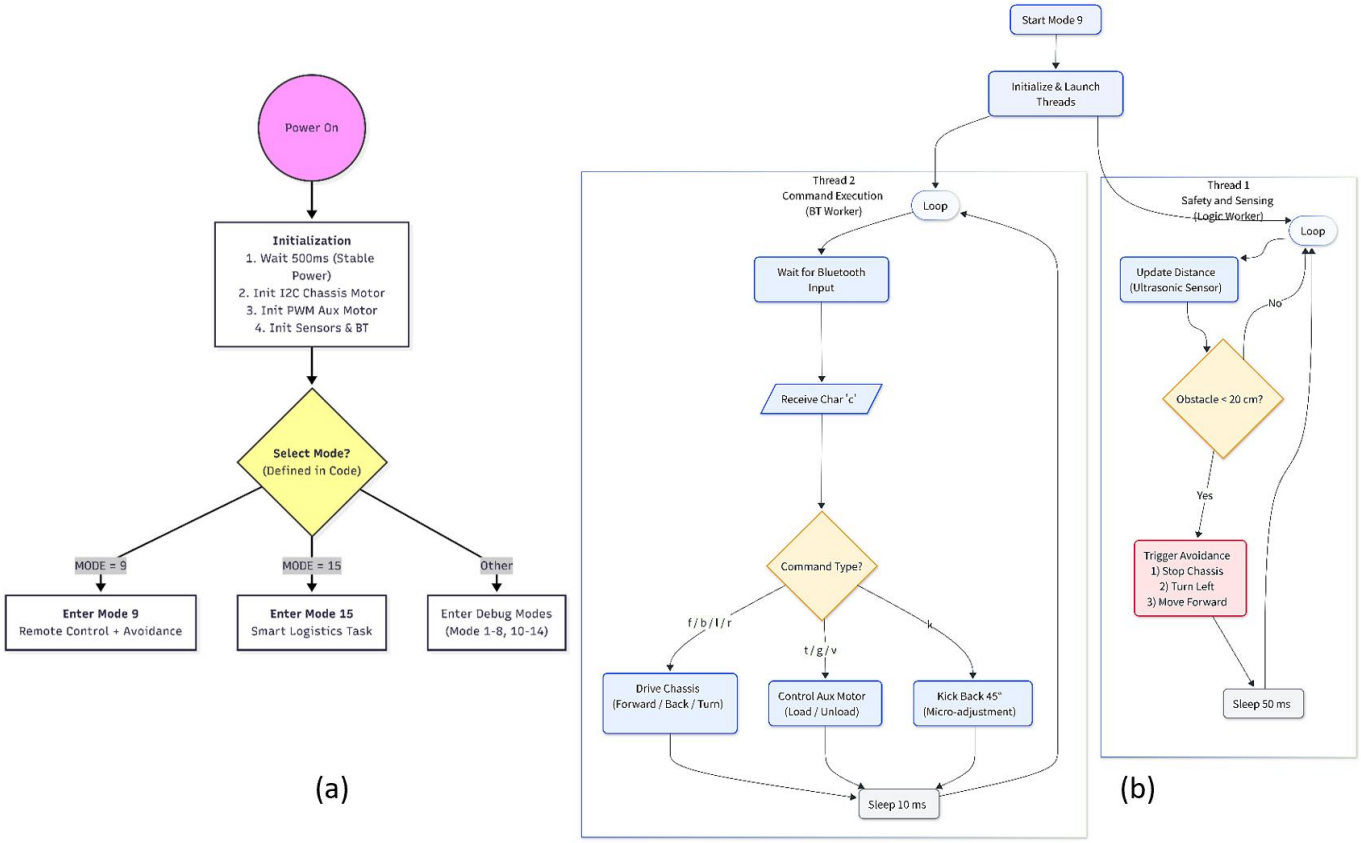

Figure 18. (a) The main flow diagram and (b) The remote control and obstacle avoidance workflow of Mode 9.

Figure 19 depicts the flowchart for the Smart Logistics Task of Mode 15. Upon activation, Mode 15 defaults to the IDLE (Idle) state, where manual control functionality is enabled. The system continuously monitors user input and transitions to the corresponding operational state based on distinct commands.

When the "1" command is triggered, the state machine switches to the **Auto Follow state**. Subsequently, it executes differentiated actions based on distance measurements: rapid forward movement (if the front distance exceeds 30 cm), maintenance of the current distance (if the distance is between 15 cm and 30 cm, inclusive), or backward movement (if the distance is less than 15 cm). When the "3" command is triggered, the system enters the **Recording state**. Here, it first clears the existing log and initiates a timer, then monitors all manual operations. Each time a chassis movement or auxiliary motor action is performed, the corresponding command and its duration are recorded into a memory array. When the "5" command is triggered, the system transitions to the **Returning state** and executes the path replay algorithm: it first reads the recorded array in reverse order, then inverts all direction commands (e.g., a recorded "forward for 1 second" is converted to "backward for 1 second"), and executes each command with its original duration. Upon task completion, the system automatically returns to the IDLE state, restoring the manual control mode.

In order to realize the above functionalities, three fundamental technical approaches are embraced by the implementation level of the code. First, a multithreading and mutex system is adopted, in which the parallel threads are created to ensure that there is no issue of data conflict caused by a variety of threads using the IIC bus or serial port at the same time; only after it acquires the mutex lock, instructions may be sent to the motors. Second, the resolution to the motor control stuttering problem induced by the early blocking ultrasonic ranging method is the use of a non-blocking sensor reading technique, where the ranging logic is moved to a dedicated logic consumer thread and the measurement results are critical-section-freey copied into a common global variable under the real-time domain. The main control loop only needs to read this variable to obtain the latest distance, thus achieving a zero-latency response for motor control. Finally, to address the bus hang-up issue that may occur on IIC devices (chassis driver board) at the moment of power-on, a dynamic object initialization scheme is adopted: the initialization of the DCMotor object is delayed until

500 milliseconds after the execution of the main function, and the object is dynamically created using the new keyword to ensure stable startup of the device.

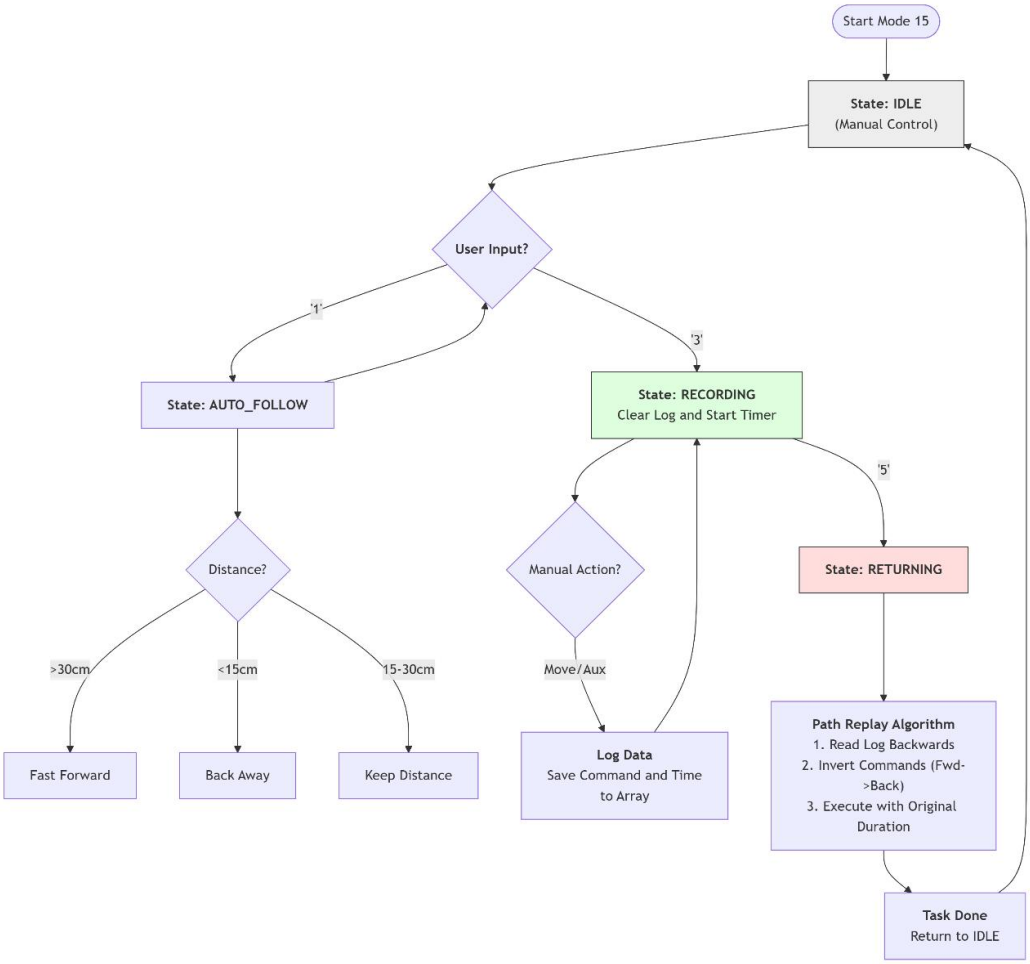

Figure 19. The flowchart for the Smart Logistics Task of Mode 15.

6.3 Bluetooth Setting

In the project, the HC-08 Bluetooth module is selected as the core wireless communication unit of the trolley, and its selection and integration directly determine the comprehensive performance of the wireless control link. Based on Bluetooth Low Energy (BLE) technology, the HC-08 complies with the IEEE 802.15.1 standard and Bluetooth 4.0+ protocol stack, and adopts the GATT/ATT architecture for data interaction. Compared with the HC-05 module, it has significantly reduced power consumption and optimized communication performance. The UART transparent transmission mode enables bidirectional data transmission between the module and the mbed LPC1768 main control board once a physical connection is established. In terms of communication, the module operates in the 2.4 GHz frequency band, adopts the GFSK modulation method, and its 30-meter communication distance can meet the requirements of actual operating conditions; in terms of hardware compatibility, its operating voltage of 3.3 V-5 V is fully compatible with the signal-level power supply of the main control board and sensors, eliminating the need for an additional voltage conversion circuit, and its compact package of 15 mm×25 mm also facilitates integration into the PCB layout of the top acrylic plate.

Based on the communication characteristics of the HC-08 module, this project selects the "HC Bluetooth Assistant" APP on the Android platform as the interactive interface; this software supports full-process visualization of command issuance and status feedback, and the Bluetooth connection operation is extremely convenient. Turn on the Bluetooth and location permissions of the Android device, then launch the "HC Bluetooth Assistant" APP; meanwhile, when the power supply of the trolley is turned on, the HC-08 module automatically enters the discoverable state, and its indicator light turns blue. Locate the "HC-08" module in the device list of the APP, click to connect; when "Connected" is displayed at the top of the page, the communication link is fully established, allowing command interaction and control operations to be performed.

Based on the communication characteristics of the HC-08 module, this project adopts the "HC Bluetooth Assistant" APP on the Android platform as the interactive interface. The top of the APP is the device connection area, which displays the Bluetooth connection status in real time (Disconnected/Connected), automatically scans surrounding devices and marks the exclusive name of the HC-08 module (preset as "IntelligentLogisticsCar-HC08") to facilitate quick identification, and supports one-click connection and disconnection. The middle part is the command input and data received section, a single character input box, set of quick command buttons and a send key, which provides the real time feedback of the status of the trolley (ultrasonic detection distance, motor velocity, and tipping bucket working state) and logs the time and frequency of data transfer, and provides the functionality to clear and save locally to enable future traceability and analysis of experimental data. The bottom part is the custom key area, aiming at the core control requirements of the trolley (such as forward/backward movement, steering, acceleration/deceleration, tipping bucket unloading, etc.), users can long-press to edit preset quick commands, input custom button names and corresponding control characters (e.g., "Tipping Bucket Lift" corresponds to "U"), and realize one-click activation of commonly used functions after saving to improve operation efficiency. This UI application is very compatible with the UART transparent transmission system of UC-08 module, and short-frame commands transmit allows the trolley to react swiftly and satisfy the needs of sophisticated operations.

The following Table 5 lists the commands in the program, their corresponding actual operations, and accompanying function explanations (the speed in the table indicates the duty cycle, which is much clearer).

Table 5. Commands in the program, corresponding operations and explanations.

Command	Operation	Explanation
f	Forward	Initial speed =20; left/right wheels rotate forward synchronously
b	Backword	Initial speed =20; left/right wheels rotate backward synchronously
a	Accelerate	Add 5 to base speed; upper limit =80
d	Decelerate	Subtract 5 from current speed; lower limit =0
l	Turn Left	Add 5 to steering value; upper limit =60
r	Turn Right	Subtract 5 from steering value; lower limit =60
z	Smooth Stop	Smoothly transition from a moving state to complete rest
s	Stop	Drive motors stop running
t	Tipping Bucket Motor Forward Rotation	Runs until stop command, speed =1
g	Tipping Bucket Motor Reverse Rotation	Runs until stop command, speed =1
v	Tipping Bucket Motor Stop	-
1	Auto Follow	Follow when >20cm from object; retreat when <20cm
3	Start Path Recording	Record order "A-B-C-D"
4	Replay Recorded Path	Replay order "A-B-C-D"
5	Reverse Replay Recorded Path	Execute order "D-C-B-A"
x	Stop Current Mode	-

7 Results and Discussion

The testing of the project followed a hierarchical “component-first, system-later” workflow. By combining simulation analysis and physical testing, the rationality and functional feasibility of the hardware design were comprehensively verified, while key technical issues encountered during development were addressed in a targeted manner. Ultimately, all intended design objectives were achieved. The following outlines the main issues, solutions, and final outcomes for each development segment.

In the mechanical design of the vehicle, testing also ascertained the functionality of the hardware system to perform the desired design functions. The first design had the addition of a limiter underneath the tipping bucket to ensure that the reset could only be done too much when operating manually. This component was however removed later- its presence significantly reduced operation tolerance and increased the risk of tooth scuffing between the two gears. The dual-

power supply was cunningly fixed on and fitted with the studs and nylon straps; the frontrear mounting system not only made charging and wiring easy, but also, ideal weight proportion ratio between the front and rear. The frame achieved both the lightweight and the rigidity goals: the safety factor of the structural strength in ANSYS simulation was over 4.0, and the frame demonstrated no loosening and misalignment in the impact of minor impacts. The mechanical framework of the vehicle was also stable and the unloading system could carry out full-load handling tasks steadily. Mecanum wheels made it possible to move in all directions gone, and the accuracy of the motor control was high enough to suit even complicated routes of work. The layout of the PCB and circuit wiring was rational which made the transmission of signals stable and without any electromagnetic interference, and the wire connections were clean and organized. All the essential pointers passed the compliance requirements and standards of the design.

The general layout of PCB was right. The SDA, and SCL versions of the IIC interface, however, were of open-drain output architecture, and hence cannot source out a high level on its own, hence a pull-up resistor is necessary to hold the high state. In initial experimentation, it was found that with the mbed inserted in its corresponding expansion board pin 27 and pin 28 of the mbed could allow the transmission of normal IIC signals without extra pull-up resistors. This resulted in a false assumption of the internal circuitry of the mbed of providing built-in pull-up resistors in pin 27 and 28, which leads to the false design of PCB without pull-up resistors. On formal experiments, though, the PCB did not work well. Various checks and balances led to a conclusion that the pull-up resistors to the mbed were in the expansion board. Two external pull-up resistors were then connected through a breadboard in order to correct the miss of pull-up resistors in PCB. With this modification, the PCB was capable of entirely replacing the unnecessary breadboard and mbed expansion board, and it was capable of running in its usual state and could operate all the functions of the vehicle.

During the software development process, multiple technical bottlenecks were encountered sequentially due to issues such as the IIC protocol, and all were resolved eventually. During the initial hardware connection, pin 9 and pin 10 were selected as SDA and SCL, but the motor showed no response at all. After reviewing the mbed LPC1768 manual and schematic, the root cause was identified: these pins did not integrate default pull-up resistors, which caused the line to float and timing disorder when the IIC bus output logic "1". By relocating the interface to pin 28/27 (pre-equipped with pull-up resistors), the physical layer communication link was successfully established. Afterward with the IIC clock rate of the PCA9685 adjusted to 400 kHz, there were temporary stutters and loss of commands with the motor in the high load mode. It was found that due to high frequency electromagnetic noise produced as a result of commutation in themselves of the brushed motor, it was likely to cause interference to the high-frequency signals of narrow pulses. In this way, the frequency was anticipatorily reduced to 100 kHz and the extended signal cycle had improved the system strength. During the addressing phase, the 7-bit address 0x34 specified in the PCA9685 manual did not match the 8-bit address required by the mbed API. By shifting this address left by one bit to obtain 0x68, the master-slave device handshaking issue was resolved. For the bus deadlock caused by the slow initialization of the driver board during power-on, a dynamic initialization mechanism and retry logic were introduced. Error detection and automatic retries were added to the DCMotor class, which completely eliminated the instability of the IIC. Concurrently in the single-threaded architecture, ultrasonic ranging was blocking wait, which consumed CPU resources, causing Bluetooth command traffic to build up and go dead. To solve this, a multi-threaded architecture of the RTOS was added: a high-priority task is assigned to scanning Bluetooth commands, a background one to perform more HEAVY ranging and obstacle avoidance logic. With this decoupling design, perception and control are performed in parallel and system latency is removed altogether.

After testing at all phases, the functions and performance of the software system have fully met the expected requirements. All Modes 1-8 in the unit verification phase stably realize their corresponding functions. During the subsystem integration phase, the automatic obstacle avoidance and omnidirectional remote control feature timely response and reliable logic. Mode 15, the system-level application, fully integrates all functions: the path memory can accurately reproduce complex movements, and the smooth stop algorithm effectively mitigates inertial impact. After the implementation of the multi-threaded architecture, the response latency of Bluetooth commands is controlled at the millisecond level, and there is no lag in remote control operations even when the ultrasonic sensor is in the ranging waiting state. After optimization of pins, frequency, address, and other parameters, the IIC communication remains stable under high-load operating conditions of the motor, with no data loss or timing disorder, and all functions are successfully achieved, which is indeed remarkable. Ultimately, the software system achieves efficient collaboration between perception, control, and interaction, satisfying the practical application requirements of the intelligent logistics vehicle.

8 Future Work

In summary, although all expected functions have been achieved, the project currently exists as a small-scale verification model. Future iterations will focus on optimizing it for practical engineering applications, which requires specific improvements and functional expansions.

With respect to the mechanical structure, the tipping bucket drive may be substituted by the hydraulic structure or the high-reliability gearbox to satisfy the heavy-load demand of the practical engineering. The geometric structure of the tipping bucket is to be optimized: it will be extended and placed in the centre of the vehicle body to minimize the deviation of the centre-of-gravity, and more brackets will be provided on both sides of the tipping bucket to prevent the movement to the left or right. The frame design will also be allocated a special battery compartment that will substitute the nylon strap attachment hence enhancing stability of the structure. Also, suitable length wires will be taken to each interface to bring about tidy wiring arrangement.

To increase the hardware, the gravity sensor and LCD screen will be included to achieve the concept of real-time graphical display of the load weight. A buzzer and an LCD interactive screen will also be installed to cause audio-visual alerts when the machine is located in the working conditions like unloading, turning, and obstacle detection to improve human-machine contact and human-machine safety. Regarding functions and algorithms, the primary focus will be made on the upgrading of obstacle avoidance system. The existing one approach of turning on an opposing direction whenever faced with a challenge does not suit complex settings. These will be further increased in the future by adding more sets of ultrasonic sensors, and a multi-path planning algorithm is to be generated so that dynamic obstacle avoidance and path optimization are realized and the passability in severe situations can be improved.

The aforementioned improvements will enhance the engineering practicality of the project and promote its transformation from a verification model to a practically usable intelligent logistics transport equipment.

9 Conclusion

This project aims to develop a multifunctional intelligent logistics transport vehicle based on the mbed LPC1768 microcontroller, so as to improve the automation level of short-distance material handling. The vehicle design adopts a modular hardware architecture and a software system based on Finite State Machine (FSM), striving for structural compactness and functional integration. The reliability of the mechanical structure was verified through SolidWorks modeling and ANSYS simulation analysis: the simulation results show that under the rated 2kg load, the maximum deformation is only 0.13 mm, the maximum stress is 9.99 MPa, and the safety factor is 4.40, indicating that the structural strength meets the design requirements. Physical assembly and testing further confirmed the lightweight and high rigidity of the frame—there is no structural loosening under minor impacts, and the tipping bucket unloading mechanism can stably carry out operations under the rated load. In addition, four omnidirectional Mecanum wheels enable the vehicle's omnidirectional movement, and their driving accuracy meets the operational requirements of complex preset trajectories.

The team optimized the circuit layout in the electrical design and developed the PCB in the PCB design. At the integrated testing stage, it was found that the problem was the absence of pull-up resistors on the IIC bus, which the addition of pull-up resistors to respective pin 27 and pin 28 managed to solve successfully. The PCB has substituted the breadboard and the expansion board in the end, and the wiring of the circuit is compact in size without causing issues with the signal transmission, and all the units of a circuit are functional.

In terms of software control, modular design and multi-threaded RTOS architecture are adopted to ensure the parallel execution of tasks such as ultrasonic ranging and Bluetooth command processing. After phased debugging, each functional module operates stably: in the integrated test, the vehicle can accurately reproduce the recorded path, automatic obstacle avoidance responds timely and reliably, the smooth stop algorithm effectively suppresses inertial impact, and Bluetooth remote control responds quickly without significant delay. Overall, all predetermined functions of the intelligent logistics vehicle (movement along preset trajectories, Bluetooth remote control, ultrasonic obstacle avoidance and parking, auto-follow, tipping bucket unloading, etc.) have been realized and verified. The original objectives have been achieved, verifying the effectiveness of the design scheme and providing valuable reference for subsequent related research.

List of Reference

[1] J. -f. Yan and Y. -f. Li, "Research on Countermeasures to Development Trend of Intelligent Logistics and Accelerative Development of Regional Logistics Industry," *Logistics Engineering and Management*, no. 2, pp. 20-22, 2022.

[2] C. X. Xi and D. Luo, "The logistics of China ushers in a time of transformation, with innovation driving and overseas expansion the 'dual engines', reshaping the future of the industry," *Logistics & Material Handling*, no. 1, pp. 32-45, 2025.

[3] C. -m. Ji, B. Yan, and P. -q. Yang, "New Technology Application and Development Trend of Heavy Equipment Transport Vehicles," *Auto Application*, no. 6, pp. 33-34, 2014.

[4] X. Zhang, "Research and Implementation of Key Technologies for IoT Embedded Systems Based on mbed," M.S. thesis, Univ. of Chinese Acad. of Sci. (Shenyang Inst. of Comput. Technol., Chinese Acad. of Sci.), Beijing, China, 2019.

[5] J. Sun, Y. Zhao, S. Wang, X. Wang, and Y. Zhang, "Design of a Desktop Cleaning Vehicle Based on Multi-Sensor Fusion," 2025, doi: 10.16589/j.cnki.cn11-3571/tn.2025.12.004.

[6] M. Xue, "Optimization Design and Performance Analysis of Wireless Communication Protocol Stack Based on Finite State Machine," 2025, doi: 10.19772/j.cnki.2096-4455.2025.09.037.

[7] J. Li and S. Guo, "A kind of 'Smooth Stop' Sensor System Integrated into Front Calipers of New Energy Vehicles," Chinese Patent Appl. Publ. CN 120986359 A, Nov. 21, 2025.

Appendix

The entire source code of the intelligent logistics transport vehicle project has been provided in this Appendix. Code Development All the code takes the mbed LPC1768 microcontroller, developed in C language, and follows the embedded system programming rules to the letter.

The program coding has been tested with the use of physical hardware, and with complete implementation of all the software functions that were outlined in the body of the report. It solves technical problems, including IIC pin adaptation, clock frequency optimization, address matching. Comments are also stored in the code, with the purpose of each function well indicated, the definition of each parameter, and important logic implementation information, making this easier to refer to and development secondary.

DCMotor.h

```
#undef __ARM_FP

#ifndef DCMOTOR_H

#define DCMOTOR_H



#include "mbed.h"

#include <cstdint>



// 4-Channel I2C DC motor driver

// Speed range: -100 .. +100

class DCMotor {

public:

    // I2C Address (7-bit)
```

```
static constexpr uint8_t I2C_ADDR_7BIT           = 0x34;


// Register Definitions
static constexpr uint8_t REG_MOTOR_TYPE          = 0x14;
static constexpr uint8_t REG_ENCODER_POLARITY    = 0x15; // Affects direction/controllability
static constexpr uint8_t REG_MOTOR_FIXED_SPEED   = 0x33; // Fixed speed control
static constexpr uint8_t MOTOR_TYPE_JGB37_520_12V  = 0x03; // Motor Type: 110RPM


DCMotor(PinName sda, PinName scl, uint8_t addr7bit = I2C_ADDR_7BIT);


// --- Basic APIs ---
// Set speed for Motor 1 only
void setSpeed(int8_t m1_speed);


// Set speed for all 4 motors individually
void setMotors(int8_t m1, int8_t m2, int8_t m3, int8_t m4);


// Set speed for Left (M1, M3) and Right (M2, M4) sides
void setLR(int8_t left, int8_t right);


// Stop all motors immediately
void stopAll() { setMotors(0, 0, 0, 0); }


// --- Robust APIs (Recommended for stability) ---
// Configure motor parameters with retry mechanism
bool configure(uint8_t motor_type = MOTOR_TYPE_JGB37_520_12V,
               uint8_t encoder_polarity = 0x00,
               int retries = 5);


// Force polarity register to 0 to prevent uncontrollable spinning
bool ensurePolarity0(int retries = 5);


// Set motors with ACK check and retries
bool setMotorsChecked(int8_t m1, int8_t m2, int8_t m3, int8_t m4, int retries = 3);
bool setLRChecked(int8_t left, int8_t right, int retries = 3);
```

```
    // Debug helper: Read 8-bit register
    bool readReg8Checked(uint8_t reg, uint8_t &value, int retries = 3);

private:
    // Low-level I2C write with retry logic
    bool writeBytesChecked(const char *data, int len, int retries);
    bool writeReg8Checked(uint8_t reg, uint8_t value, int retries);

private:
    I2C _i2c;
    int _addr; // 8-bit address used by mbed I2C API (7-bit << 1)
};

#endif
```

DCMotor.cpp

```
#undef __ARM_FP
#include "DCMotor.h"
#include <chrono>

using namespace std::chrono;

DCMotor::DCMotor(PinName sda, PinName scl, uint8_t addr7bit)
    : _i2c(sda, scl), _addr(addr7bit << 1)   // Shift 7-bit address to 8-bit
{
    // Lower frequency to 100kHz to improve stability against motor noise
    _i2c.frequency(100000);

    // Wait for the motor driver board to power up fully
    ThisThread::sleep_for(200ms);

    // Attempt configuration (Best effort)
    (void)configure(MOTOR_TYPE_JGB37_520_12V, 0x00, 5);
```

```
    // Ensure motors are stopped on startup
    stopAll();
}

bool DCMotor::writeBytesChecked(const char *data, int len, int retries)
{
    for (int i = 0; i < retries; i++) {
        // _i2c.write returns 0 on success (ACK received)
        int ret = _i2c.write(_addr, data, len, false);
        if (ret == 0) return true;

        // Wait briefly before retrying
        ThisThread::sleep_for(2ms);
    }
    return false;
}

bool DCMotor::writeReg8Checked(uint8_t reg, uint8_t value, int retries)
{
    char buf[2];
    buf[0] = static_cast<char>(reg);
    buf[1] = static_cast<char>(value);
    return writeBytesChecked(buf, 2, retries);
}

bool DCMotor::configure(uint8_t motor_type, uint8_t encoder_polarity, int retries)
{
    bool ok1 = writeReg8Checked(REG_MOTOR_TYPE, motor_type, retries);
    ThisThread::sleep_for(5ms);
    bool ok2 = writeReg8Checked(REG_ENCODER_POLARITY, encoder_polarity, retries);
    ThisThread::sleep_for(5ms);
    return ok1 && ok2;
}

bool DCMotor::ensurePolarity0(int retries)
```

```
{
    // Critical safety: Force Register 0x15 to 0.
    // If set to 1, motors may spin uncontrollably.
    bool ok = writeReg8Checked(REG_ENCODER_POLARITY, 0x00, retries);
    ThisThread::sleep_for(2ms);
    return ok;
}

bool DCMotor::setMotorsChecked(int8_t m1, int8_t m2, int8_t m3, int8_t m4, int retries)
{
    char data[5];
    data[0] = static_cast<char>(REG_MOTOR_FIXED_SPEED);
    data[1] = static_cast<char>(m1);
    data[2] = static_cast<char>(m2);
    data[3] = static_cast<char>(m3);
    data[4] = static_cast<char>(m4);
    return writeBytesChecked(data, 5, retries);
}

bool DCMotor::setLRChecked(int8_t left, int8_t right, int retries)
{
    // Map Logic: Left = (M1, M3), Right = (M2, M4)
    return setMotorsChecked(left, right, left, right, retries);
}

// ---- Legacy API wrappers ----
void DCMotor::setMotors(int8_t m1, int8_t m2, int8_t m3, int8_t m4)
{
    (void)setMotorsChecked(m1, m2, m3, m4, 1);
}

void DCMotor::setSpeed(int8_t m1_speed)
{
    setMotors(m1_speed, 0, 0, 0);
}
```

```
void DCMotor::setLR(int8_t left, int8_t right)
{
    setMotors(left, right, left, right);
}

// Debug: Read register (if hardware supports)
bool DCMotor::readReg8Checked(uint8_t reg, uint8_t &value, int retries)
{
    char r = static_cast<char>(reg);
    for (int i = 0; i < retries; i++) {
        int ret = _i2c.write(_addr, &r, 1, true); // Repeated start
        if (ret != 0) {
            ThisThread::sleep_for(2ms);
            continue;
        }
        char v = 0;
        ret = _i2c.read(_addr, &v, 1, false);
        if (ret == 0) {
            value = static_cast<uint8_t>(v);
            return true;
        }
        ThisThread::sleep_for(2ms);
    }
    return false;
}
```

main.cpp

```
#undef __ARM_FP
#include "mbed.h"
#include "DCMotor.h"
#include <cstring>
#include <cstdint>
#include <cstdio>
#include <vector>
```

```
#include <chrono>

#include <cmath>


using namespace std::chrono;


// ================== 1. Hardware Interface Definitions ==================


// PC Serial Port (USB)

BufferedSerial pc(USBTX, USBRX, 9600);


// Bluetooth Serial Port (UART)

UnbufferedSerial bt(p9, p10, 9600);

DigitalOut bt_led(LED1);


// Pointer to I2C Motor Driver Object

DCMotor *motorBoard = nullptr;


// Ultrasonic Sensor (GPIO + Timer)

DigitalOut trig(p11);

DigitalIn  echo(p12, PullDown);

Timer      sonar_timer;

Mutex      sonar_mtx;


// Auxiliary Motor (PWM Control: p21, p22)

PwmOut aux_p1(p21);

PwmOut aux_p2(p22);


// Time duration for 45-degree rotation simulation

#define FIXED_ACTION_MS  150


// Global Variable for Distance

volatile float g_latest_dist = 999.0f;


// ================== 2. Helper Functions ==================
```

```
void pc_puts(const char *s) { pc.write(s, strlen(s)); }
inline bool is_ignorable_char(char c) { return (c == '\r' || c == '\n'); }

// Blocking character read from PC
char pc_getc_blocking() {
    char c;
    while (pc.read(&c, 1) <= 0) { }
    return c;
}

// ================== 3. Auxiliary Motor Control Class ==================
struct AuxControl {
    float speed = 0.01f;
    int dir = 0;        // 0:Stop, 1:Forward, -1:Reverse

    AuxControl() {
        // Set PWM period to 20ms (50Hz) for standard DC motor control
        aux_p1.period_ms(20);
        aux_p2.period_ms(20);
    }

    void update() {
        if (dir == 0) {
            aux_p1.write(0.0f); aux_p2.write(0.0f);
        } else if (dir == 1) {
            aux_p1.write(speed); aux_p2.write(0.0f);
        } else if (dir == -1) {
            aux_p1.write(0.0f); aux_p2.write(speed);
        }
    }

    // Micro-adjustment: Briefly kick the motor to prevent jamming
    void kick_back_45() {
        aux_p1.write(0.0f); aux_p2.write(speed);
        ThisThread::sleep_for(milliseconds(FIXED_ACTION_MS));
```

```
        aux_p1.write(0.0f); aux_p2.write(0.0f);
        dir = 0;
    }

    void cmd(char c) {
        if (c == 't') { dir = 1; update(); } // Tipping (Unload)
        else if (c == 'g') { dir = -1; update(); } // Retract
        else if (c == 'v') { dir = 0; update(); } // Stop

        // Speed Control
        else if (c == 'y') { speed += 0.02f; if(speed > 1.0f) speed = 1.0f; update(); }
        else if (c == 'h') { speed -= 0.02f; if(speed < 0.01f) speed = 0.01f; update(); }

        else if (c == 'k') { kick_back_45(); }
    }

    void get_status(char* buf) {
        sprintf(buf, "Aux:[%s %.2f]", dir==0?"STOP":(dir==1?"FWD":"REV"), speed);
    }
};

AuxControl aux;

// ================== 4. Chassis Control Wrapper (with Soft Stop) ==================
struct ChassisControl {
    int8_t last_L = 0;
    int8_t last_R = 0;

    void set(int8_t left, int8_t right) {
        if (motorBoard == nullptr) return;
        if (left != last_L || right != last_R) {
            motorBoard->setLR(left, right);
            last_L = left;
            last_R = right;
        }
```

```
    }

    // Hard Stop (Emergency Stop)
    void stop() { set(0, 0); }

    // [Core Algorithm] Linear Soft Stop
    // Simulates constant deceleration to prevent cargo tipping.
    // k_step: Deceleration rate (Higher = Faster stop).
    void smooth_stop(int8_t k_step = 5, int interval_ms = 50) {
        if (motorBoard == nullptr) return;
        if (last_L == 0 && last_R == 0) return;

        while (last_L != 0 || last_R != 0) {
            // Linear ramp down for Left Wheel
            if (last_L > 0) { last_L -= k_step; if (last_L < 0) last_L = 0; }
            else if (last_L < 0) { last_L += k_step; if (last_L > 0) last_L = 0; }

            // Linear ramp down for Right Wheel
            if (last_R > 0) { last_R -= k_step; if (last_R < 0) last_R = 0; }
            else if (last_R < 0) { last_R += k_step; if (last_R > 0) last_R = 0; }

            motorBoard->setLR(last_L, last_R);
            ThisThread::sleep_for(milliseconds(interval_ms));
        }
        stop(); // Ensure final state is exactly zero
    }
};
ChassisControl chassis;

// ================== 5. Ultrasonic Sensor Driver ==================
void update_distance_reading() {
    sonar_mtx.lock();
    // Trigger Pulse (10us)
    trig = 0; wait_us(2);
    trig = 1; wait_us(10);
```

```
    trig = 0;


    sonar_timer.reset();
    int timeout = 0;


    // Wait for Echo High
    while (echo == 0) {
        if (++timeout > 20000) { sonar_mtx.unlock(); g_latest_dist = 999.0; return; }
        wait_us(1);
    }


    // Measure Pulse Width
    sonar_timer.start();
    timeout = 0;
    while (echo == 1) {
        if (++timeout > 50000) break;
        wait_us(1);
    }
    sonar_timer.stop();
    sonar_mtx.unlock();


    // Calculate Distance (cm)
    long long pulse_us = duration_cast<microseconds>(sonar_timer.elapsed_time()).count();
    float d = (float)pulse_us / 58.0f;
    if (d <= 0.1f) d = 999.0f;
    g_latest_dist = d;
}


// ================== Debug Modes (Modes 1-8, 10-14) ==================
// Unit tests for individual components
void debug_single_motor() { pc_puts("[M1] Single Motor\r\n"); while(1) { char c=pc_getc_blocking();
if(c=='f') motorBoard->setSpeed(50); else if(c=='b') motorBoard->setSpeed(-50); else motorBoard-
>setSpeed(0); } }
void debug_auto_square() { pc_puts("[M2] Auto Square\r\n"); chassis.set(50,50);
ThisThread::sleep_for(5s); chassis.stop(); while(1) ThisThread::sleep_for(1s); }
void debug_ramp_test() { pc_puts("[M3] Ramp Test\r\n"); int8_t s=0; chassis.stop(); while(s<80){s+=5;
chassis.set(s,s); ThisThread::sleep_for(500ms);} while(1) ThisThread::sleep_for(1s); }
```

```
void debug_front_rear_serial() { pc_puts("[M4] F/R Serial\r\n"); while(1) { char
c=pc_getc_blocking(); if(c=='f') motorBoard->setLR(50,0); else if(c=='r') motorBoard->setLR(0,50);
else chassis.stop(); } }

void debug_front_rear_auto() { pc_puts("[M5] F/R Auto\r\n"); while(1) { motorBoard->setLR(50,0);
ThisThread::sleep_for(3s); motorBoard->setLR(0,50); ThisThread::sleep_for(3s); } }

void debug_circle_auto() { pc_puts("[M6] Circle Auto\r\n"); while(1) { chassis.set(50,50);
ThisThread::sleep_for(3s); chassis.set(0,50); ThisThread::sleep_for(2s); } }

void debug_all_wheels_serial_timed() { pc_puts("[M7] All Serial\r\n"); while(1)
{ if(pc_getc_blocking()=='g') { chassis.set(50,50); ThisThread::sleep_for(5s); chassis.stop(); } } }

void bt_led_mode() { pc_puts("[M8] BT LED\r\n"); while(1) { char c; if(bt.read(&c,1)>0) bt_led =
(c=='1'); } }

void debug_all_motors_serial_fb() { pc_puts("[M10] All Serial FB\r\n"); while(true) { char c =
pc_getc_blocking(); if (c=='f') motorBoard->setMotors(50,50,50,50); else if (c=='b') motorBoard-
>setMotors(-50,-50,-50,-50); else if (c=='s') motorBoard->stopAll(); } }

void pc_diff_drive_mode() { pc_puts("[M11] Diff Drive\r\n"); while(1){ char c=pc_getc_blocking();
if(c=='f') chassis.set(50,50); else chassis.stop(); } }

void pc_speed_ramp_mode() { pc_puts("[M12] Speed Ramp\r\n"); int s=0; while(1){ char
c=pc_getc_blocking(); if(c=='a') s+=10; if(c=='s') s=0; chassis.set(s,s); } }

void pc_diff_ramp_drive_mode() { pc.set_blocking(false); pc_puts("[M13] PC Control+Avoid\r\n");
int8_t base=0, dir=1; while (true) { update_distance_reading(); if (g_latest_dist > 0.1f &&
g_latest_dist < 20.0f) { chassis.stop(); ThisThread::sleep_for(500ms); chassis.set(-50,50);
ThisThread::sleep_for(600ms); chassis.set(40,40); base=40; dir=1; } if (pc.readable()) { char c; if
(pc.read(&c, 1) > 0 && !is_ignorable_char(c)) { if(c=='f') { dir=1; base=40; chassis.set(40,40); }
else if(c=='s') { base=0; chassis.stop(); } } } ThisThread::sleep_for(20ms); } }

void auto_loop_mode() { pc_puts("[M14] Auto Loop\r\n"); while(1) { chassis.set(50,50);
ThisThread::sleep_for(2s); chassis.stop(); ThisThread::sleep_for(2s); } }
```

```
// ================== Mode 9: Remote Control & Obstacle Avoidance ==================

namespace mode9

{

    static Thread g_bt_thread(osPriorityNormal, 1024);

    static Thread g_logic_thread(osPriorityNormal, 1024);

    static Mutex g_ctrl_mtx;


    static const int8_t SPEED_START = 20;

    static const int8_t SPEED_STEP  = 5;

    static const int8_t SPEED_LIMIT = 80;

    static const int8_t STEER_STEP  = 5;

    static const int8_t STEER_LIMIT = 60;


    static int8_t base = 0, dir = 1, steer = 0;
```

```
    static inline int8_t clamp_i8(int v, int lo, int hi) { if(v<lo)v=lo; if(v>hi)v=hi; return
(int8_t)v; }


    // Differential Drive Calculation

    static void apply_chassis() {

        int left = dir*base - steer; int right = dir*base + steer;

        left = clamp_i8(left, -100, 100); right = clamp_i8(right, -100, 100);

        chassis.set((int8_t)left, (int8_t)right);

    }


    static void handle_cmd(char cmd, const char *src_tag) {

        ScopedLock<Mutex> lock(g_ctrl_mtx);


        if(cmd=='t'||cmd=='g'||cmd=='v'||cmd=='y'||cmd=='h'||cmd=='k') { aux.cmd(cmd); }

        else if(cmd=='f') { dir=1; if(base==0)base=SPEED_START; apply_chassis(); }

        else if(cmd=='b') { dir=-1; if(base==0)base=SPEED_START; apply_chassis(); }

        else if(cmd=='a') { if(base==0)base=SPEED_START; else
base=clamp_i8(base+SPEED_STEP,0,SPEED_LIMIT); apply_chassis(); }

        else if(cmd=='d') { base=clamp_i8(base-SPEED_STEP,0,SPEED_LIMIT); if(base==0){steer=0;
chassis.stop();} else apply_chassis(); }

        else if(cmd=='l') { steer=clamp_i8(steer+STEER_STEP,-STEER_LIMIT,STEER_LIMIT);
if(base==0)base=SPEED_START; apply_chassis(); }

        else if(cmd=='r') { steer=clamp_i8(steer-STEER_STEP,-STEER_LIMIT,STEER_LIMIT);
if(base==0)base=SPEED_START; apply_chassis(); }

        else if(cmd=='z') { steer=0; if(base==0)chassis.stop(); else apply_chassis(); }


        // 's' = Hard Stop (Immediate)

        else if(cmd=='s') {

            base=0; steer=0;

            chassis.stop();

        }

        // 'z' = Soft Stop (Linear Deceleration)

        else if(cmd=='z') {

            base=0; steer=0;

            chassis.smooth_stop(6);

        }
```

```
        char aux_st[32]; aux.get_status(aux_st);

        char buf[80];

        std::snprintf(buf, sizeof(buf), "%s Cmd:%c | D:%d | %s\r\n", src_tag, cmd,
(int)g_latest_dist, aux_st);

        pc_puts(buf);

    }


    static void bt_rx_worker() { char c; while(1) { if(bt.read(&c,1)>0 && !is_ignorable_char(c))
handle_cmd(c, "[BT]"); else ThisThread::sleep_for(10ms); } }


    // Background thread for safety monitoring

    static void logic_worker() {

        while(1) {

            update_distance_reading();

            // Obstacle Avoidance Logic

            if(g_latest_dist > 0.1f && g_latest_dist < 20.0f) {

                pc_puts("!!! Avoidance Triggered !!!\r\n");

                g_ctrl_mtx.lock();


                chassis.stop();

                ThisThread::sleep_for(200ms);


                // Back up and turn

                chassis.set(-50, 50); ThisThread::sleep_for(600ms);

                dir=1; base=40; steer=0; chassis.set(40,40);

                g_ctrl_mtx.unlock();

                ThisThread::sleep_for(500ms);

            }

            ThisThread::sleep_for(50ms);

        }

    }


    void run() {

        pc.set_blocking(true); bt.set_blocking(false);

        chassis.stop(); aux.cmd('v');

        pc_puts("\r\n[Mode 9] Remote + Avoidance Ready\r\n");
```

```
        g_bt_thread.start(callback(bt_rx_worker));
        g_logic_thread.start(callback(logic_worker));
        while(1) ThisThread::sleep_for(1000ms);
    }
}

// ================== Mode 15: Smart Logistics (AGV) ==================
namespace mode15
{
    // Finite State Machine Definition
    enum State { IDLE, AUTO_FOLLOW, RECORDING, REPLAYING, RETURNING, AUTO_DOCK, SHUTTLE_RUN };
    State current_state = IDLE;

    // Path Memory Structure
    struct PathAction { char cmd; uint32_t ms; };
    PathAction path_log[100];
    int log_index = 0;
    Timer rec_timer;
    char last_rec_cmd = 's';

    static Thread g_bt_thread(osPriorityNormal, 1024);
    static Thread g_logic_thread(osPriorityNormal, 2048);
    static Mutex g_cmd_mtx;

    // Execute instant chassis command
    void execute_cmd_instant(char cmd, int8_t spd=60) {
        if(cmd=='f') chassis.set(spd, spd);
        else if(cmd=='b') chassis.set(-spd, -spd);
        else if(cmd=='l') chassis.set(-spd, spd);
        else if(cmd=='r') chassis.set(spd, -spd);
        else if(cmd=='s') {
            // Hard Stop
            chassis.stop();
        }
        else if(cmd=='z') {
```

```
            // Soft Stop (Lower k for smoother stop when loaded)
            chassis.smooth_stop(4);
        }
        else if(cmd=='t'||cmd=='g'||cmd=='v'||cmd=='k') aux.cmd(cmd);
    }

    char get_inverse_cmd(char cmd) {
        if(cmd=='f') return 'b'; if(cmd=='b') return 'f';
        if(cmd=='l') return 'r'; if(cmd=='r') return 'l';
        return cmd;
    }

    // --- Sub-function 1: Auto Docking (Closed Loop) ---
    void run_auto_docking() {
        float dist = g_latest_dist;
        if (dist > 900.0f) { chassis.stop(); return; }

        if (dist > 20.0f) { chassis.set(35, 35); }
        else if (dist > 6.0f) { chassis.set(25, 25); }
        else {
            chassis.stop(); // Precision stop required for docking
            g_cmd_mtx.lock(); current_state = IDLE; g_cmd_mtx.unlock();
            pc_puts("[AGV] Docking Complete.\r\n");
        }
    }

    // --- Sub-function 2: Smart Shuttle (Transport -> Wait -> Return) ---
    void perform_smart_shuttle() {
        pc_puts("[AGV] Shuttle Start (Sensor Based)\r\n");

        // Phase 1: Transporting to destination
        pc_puts("Phase 1: Transporting...\r\n");
        chassis.set(50, 50);

        Timer t_safety; t_safety.start();
```

```
    while (true) {
        update_distance_reading();

        // Stop Condition A: Wall detected (< 15cm)
        if (g_latest_dist < 15.0f && g_latest_dist > 0.1f) {
            chassis.stop();
            pc_puts("Phase 1: Arrived (Sensor Trigger)\r\n");
            break;
        }

        // Stop Condition B: Timeout Safety (5s)
        if (t_safety.elapsed_time() > 5s) {
            chassis.stop();
            pc_puts("Phase 1: Timeout Stop\r\n");
            break;
        }
        ThisThread::sleep_for(20ms);
    }

    // Phase 2: Unloading / Waiting
    pc_puts("[AGV] Unloading Wait (2s)...\r\n");
    ThisThread::sleep_for(2000ms);

    // Phase 3: Returning Home (Blind Reverse)
    pc_puts("Phase 2: Returning...\r\n");
    chassis.set(-50, -50);
    ThisThread::sleep_for(2500ms);

    // Phase 4: Soft Stop for Stability
    pc_puts("Phase 2: Soft Stop\r\n");
    chassis.smooth_stop(5);

    pc_puts("[AGV] Mission Done.\r\n");
    g_cmd_mtx.lock(); current_state = IDLE; g_cmd_mtx.unlock();
}
```

```
// Replay Logic
void execute_path_sequence(bool reverse_mode) {
    if (log_index == 0) {
        g_cmd_mtx.lock(); current_state = IDLE; g_cmd_mtx.unlock();
        return;
    }
    if (!reverse_mode) {
        for (int i = 0; i < log_index; i++) {
            execute_cmd_instant(path_log[i].cmd);
            ThisThread::sleep_for(milliseconds(path_log[i].ms));
        }
    } else {
        // Inverse Kinematics for Return Home
        for (int i = log_index - 1; i >= 0; i--) {
            char inv = get_inverse_cmd(path_log[i].cmd);
            execute_cmd_instant(inv);
            ThisThread::sleep_for(milliseconds(path_log[i].ms));
        }
    }
    execute_cmd_instant('s');
    g_cmd_mtx.lock(); current_state = IDLE; g_cmd_mtx.unlock();
}

// Auto Follow Logic (Hysteresis)
void run_auto_follow() {
    float dist = g_latest_dist;
    if (dist > 900.0f) chassis.stop();
    else if (dist > 30.0f) chassis.set(50,50);
    else if (dist > 25.0f) chassis.set(35,35);
    else if (dist < 10.0f) chassis.set(-45,-45);
    else if (dist < 15.0f) chassis.set(-35,-35);
    else chassis.stop();
}
```

```
    void process_command(char c) {

        g_cmd_mtx.lock();


        // Recording Logic

        bool is_valid_action =
(c=='f'||c=='b'||c=='l'||c=='r'||c=='s'||c=='z'||c=='t'||c=='g'||c=='v'||c=='k');

        if (current_state == RECORDING && is_valid_action) {

            if (c != last_rec_cmd) {

                uint32_t dt = duration_cast<milliseconds>(rec_timer.elapsed_time()).count();

                rec_timer.reset();

                if (dt > 100 && log_index < 100) {

                    path_log[log_index].cmd = last_rec_cmd;

                    path_log[log_index].ms = dt;

                    log_index++;

                }

                last_rec_cmd = c;

            }

        }


        // State switching commands

        if (c == '1') { current_state = AUTO_FOLLOW; pc_puts("[M15] Follow\r\n"); }

        else if (c == '3') { current_state = RECORDING; log_index=0; last_rec_cmd='s';
rec_timer.reset(); rec_timer.start(); pc_puts("[M15] Rec Start\r\n"); }

        else if (c == '4') {

            if(current_state==RECORDING) { uint32_t
dt=duration_cast<milliseconds>(rec_timer.elapsed_time()).count();
if(log_index<100){path_log[log_index].cmd=last_rec_cmd; path_log[log_index].ms=dt; log_index++;} }

            current_state = REPLAYING; pc_puts("[M15] Replaying\r\n");

        }

        else if (c == '5') {

            if(current_state==RECORDING) { uint32_t
dt=duration_cast<milliseconds>(rec_timer.elapsed_time()).count();
if(log_index<100){path_log[log_index].cmd=last_rec_cmd; path_log[log_index].ms=dt; log_index++;} }

            current_state = RETURNING; pc_puts("[M15] Return\r\n");

        }


        // [AGV Special Commands]

        else if (c == 'D' && current_state == IDLE) { current_state = AUTO_DOCK; pc_puts("[M15]
```

```
Docking...\r\n"); }
        else if (c == 'S' && current_state == IDLE) { current_state = SHUTTLE_RUN; } // Smart
Shuttle


        else if (c == 'x') { current_state = IDLE; chassis.stop(); aux.cmd('v'); pc_puts("[M15]
Idle\r\n"); }


        if (current_state == IDLE || current_state == RECORDING) execute_cmd_instant(c);


        g_cmd_mtx.unlock();
    }


    void bt_worker() {
        char c; while(1) { if(bt.read(&c,1)>0 && !is_ignorable_char(c)) process_command(c); else
ThisThread::sleep_for(10ms); }
    }


    void logic_worker() {
        while(1) {
            update_distance_reading();
            g_cmd_mtx.lock(); State s = current_state; g_cmd_mtx.unlock();


            if (s == AUTO_FOLLOW) { run_auto_follow(); ThisThread::sleep_for(50ms); }
            else if (s == REPLAYING) { execute_path_sequence(false); }
            else if (s == RETURNING) { execute_path_sequence(true); }
            else if (s == AUTO_DOCK) { run_auto_docking(); ThisThread::sleep_for(50ms); }
            else if (s == SHUTTLE_RUN) { perform_smart_shuttle(); }
            else { ThisThread::sleep_for(100ms); }
        }
    }


    void run() {
        pc.set_blocking(false); bt.set_blocking(false);
        chassis.stop(); aux.cmd('v');
        pc_puts("\r\n=== Mode 15: Smart Logistics ===\r\n");
        pc_puts("1:Follow | 3:Rec | 5:Ret\r\n");
```

```
        pc_puts("D:Auto-Dock | S:Smart-Shuttle | s:Stop | z:SmoothStop\r\n");


        g_bt_thread.start(callback(bt_worker));
        g_logic_thread.start(callback(logic_worker));
        while(1) ThisThread::sleep_for(1000ms);
    }
}


// ================== Mode 16: Sonar Debug Mode ==================
void sonar_debug_mode()
{
    pc_puts("\r\n=== Mode 16: Sonar Debug ===\r\n");
    chassis.stop(); aux.cmd('v');
    while (true) {
        update_distance_reading();
        char buf[64];
        if (g_latest_dist >= 900.0f) std::snprintf(buf, sizeof(buf), "Dist: Out of Range\r\n");
        else std::snprintf(buf, sizeof(buf), "Dist: %.2f cm\r\n", g_latest_dist);
        pc_puts(buf);
        bt_led = !bt_led;
        ThisThread::sleep_for(500ms);
    }
}


// ================== System Entry Point ==================
// Change this variable to select the operating mode
volatile int MODE = 1;


int main()
{
    // Wait for power stabilization to prevent I2C deadlock
    ThisThread::sleep_for(500ms);


    // Dynamic initialization
    motorBoard = new DCMotor(p28, p27);
```

```
    motorBoard->stopAll();


    pc.set_blocking(true);
    bt.set_blocking(true);


    // Mode Selector
    if (MODE == 1) debug_single_motor();
    else if (MODE == 2) debug_auto_square();
    else if (MODE == 3) debug_ramp_test();
    else if (MODE == 4) debug_front_rear_serial();
    else if (MODE == 5) debug_front_rear_auto();
    else if (MODE == 6) debug_circle_auto();
    else if (MODE == 7) debug_all_wheels_serial_timed();
    else if (MODE == 8) bt_led_mode();
    else if (MODE == 9) mode9::run(); // Remote + Avoidance
    else if (MODE == 10) debug_all_motors_serial_fb();
    else if (MODE == 11) pc_diff_drive_mode();
    else if (MODE == 12) pc_speed_ramp_mode();
    else if (MODE == 13) pc_diff_ramp_drive_mode();
    else if (MODE == 14) auto_loop_mode();
    else if (MODE == 15) mode15::run(); // Smart Logistics (FSM)
    else if (MODE == 16) sonar_debug_mode(); // Sensor Debug


    while (true) {
        ThisThread::sleep_for(1s);
    }
}
```